



Practical Design Patterns in JavaScript

Course Overview

Course Overview

Hey everyone. I'm John Mills. I'm a consultant that deals mostly with JavaScript and process and people management. I've been doing JavaScript for quite a while now and I come from a .NET and Java background. And what's always amazing to me is when I have object-oriented developers who do really good .NET and Java code, move over to JavaScript, and lose all that object-oriented knowledge and start writing very procedural JavaScript code. And in this course, I'm going to try and undo some of that and show you how to properly implement good, solid, object-oriented development styles into JavaScript. We'll take a look at objects in JavaScript and why they're different. And then we'll move into the full Gang of Four design patterns and we'll look at what those look like in the JavaScript ecosystem. We'll look at creational design patterns and structural, and then, even behavioral, just so that you can get a good solid understanding of how to adapt your JavaScript code to be much more object-oriented.

What Is a Design Pattern Anyway?

Introduction

Welcome to practical JavaScript design patterns. Now JavaScript's been around for quite a while and it's starting to kind of take over everything from the DOM in the front end, and now we've got node.js on the backend, and with Electron, and we've got JavaScript even on desktop applications. And what I want to do with this course is I want to take a deep breath and talk about some practical, real JavaScript design patterns that we can start to use to make our JavaScript a little more readable, a little more maintainable, and give us a vocabulary we can use to communicate with each other as we build out our JavaScript application. So in this course we're going to talk design patterns. We're going to talk a little about who created them, where they came from, and where they originated. What they are, what constitutes a design pattern, versus just something that we do. When do we use them? We're going to take a very practical approach. We're going to look at very specific scenarios where we would use design patterns to solve a problem. We're going to talk about where. We'll look at frontend, we're going to look at backend; we'll look all over the place. We'll do a lot of our work in Node. We'll do some work in Angular. We'll do some just plain vanilla JavaScript. I'll



show you where to use certain design patterns. And this is going to be a very practical course. We'll specifically implement these solutions in an application so that this isn't just theory, we'll actually have a real-world implementation for each of these design patterns.

The Problem

Now JavaScript's been around for quite a while. It was actually written back in 1995 by Brendan Eich, who was working at Netscape at the time. He actually wrote it in about 10 days. Now the JavaScript that you use today and the JavaScript that he wrote in 10 days are not exactly the same thing, but the origins of it are the same. And Brendan actually had some interesting things about this language that he created, and was pretty surprised with how big it became over the course of time. Now it originally was written to manage the DOM and interact with webpages on the screen, but over time JavaScript has grown up, partially due to the rise of these frameworks. So we've Knockout and BACKBONE, ember, React, Angular, and a whole bunch of other ones that are not one the screen. And now with Node.js on the backend and Electron on the desktop, we're starting to have full-blown, end-to-end applications that are being written entirely in JavaScript. And because of its origins and the problems he was trying to solve at the time it was created, it's just a little different than a lot of other languages that are out there, and JavaScript is really easy to write really bad code. And I've seen a lot of really good developers move from cool, nice, object-oriented, clean. NET or Java code, that as soon as they start writing JavaScript it all becomes just procedural, messy, spaghetti code, and so I want to spend some time and fix that for a lot of people.

Design Patterns to the Rescue

Now in this course we're going to start applying more of an object-oriented approach to the way we build JavaScript. A lot of you are coming from either a. NET or a Java background or some other object-oriented language, and objects work a little bit differently in JavaScript. So we're going to work through an object-oriented approach to how we build our JavaScript. And the way we're going to do that is with some tried and tested patterns for clean and elegant implementations. Now, the concept of design patterns comes from an architect named Christopher Alexander. Not this kind of architect; this kind of architect. In 1977, he wrote a book called A Pattern Language where he began to outline common patterns that he came across while building buildings and towns and cities, and, now before you check out and say, I don't build houses; it's not even close to the same thing as what we do in software, listen to this quote from Christopher and tell me if it kind of rings true for us. He said each pattern, so each pattern that he saw, represents our current best guess as to what arrangement of



the physical environment will work to solve the problem presented. Now let's change physical to virtual, and this kind of makes sense for us. Each of these design patterns we're going to talk about represents our current best guess as to what arrangement of code will work to solve the problem that we've presented, and we'll do specific problems with specific solutions. And the empirical question centers on the problem. Anytime we're doing design patterns at all, it always starts with a problem. What's the particular problem we're trying to solve and what design pattern will help us fix that? And the solution — does the arrangement we propose, does the code we're going to write, in fact, resolve the problem? And I'll give you some examples of some things that Christopher was dealing with, and then we'll look at some stuff that we deal with in code.

Physical Design Patterns

So ultimately it comes down to common problems and common solutions. Now the problems that Christopher dealt with were on structures and people, and this has been a little while, so I'm going to update some problems that we might face today. And these are things that you would be familiar with. So, on and off traffic for highways. So you've got two highways that are intersecting and you need to go from one highway to the other highway. The pattern that use for that is the Cloverleaf interchange. Right? It's a common pattern that you see almost everywhere, at least in the States, and that's a very common pattern for dealing with that problem. Another problem is pedestrian traffic. So I've got busy roads in a busy central town area, how do I deal with pedestrians? Well, we have sidewalks almost everywhere in the world there's sidewalks, and that's the pattern that they came up with to solve that problem. And the last one is entry and exit for public buildings. So, we want to maintain temperature in the buildings; we don't want the doors opening and closing all the time, so they came up with this thing called revolving doors, and that's the pattern that a lot of buildings now use for dealing with the problem of entry and exit. Now, the theme here is we have a specific problem we're trying to solve and a solution. The design patterns aren't just fun things that we throw in just because, hey, I want to see what happens when I put a sidewalk here. We're trying to solve a specific problem.

The Gang of Four

Enter the Gang of Four, Erich, Richard, Ralph, and John took Alexanders' concepts and turned them into this definitive guide of common patterns for software development. They wrote the book Design Patterns, and really, for all intents and purposes in the modern day, it's just called the Gang of Four book. And what they did is they provided us with some common, reasonable patterns for solving common problems that developers face. Again, the focus is on the problem, not on the pattern. So examples from theirs are things like we



want a service layer. I need a service layer that deals with all of my data manipulation, so I want to design a service layer that's going to separate all of my data-manipulation code from my controller, so I'm going to use the Module pattern. That's what that's all about. The Module pattern's going to create a little package that's going to do some work for. I've got overly complicated object interfaces. So who hasn't seen this? So let's say the DOM, for example. Very overly complicated object interfaces. So we're going to come up with the Façade Pattern. The Façade pattern is just something that's just going to sit over top of that complicated object and make things simpler. JQuery is a great example of a Façade pattern implemented over top of something that is horribly complicated, like the DOM. The last one's going to be visibility into state changes. So I've got one object that undergoes changes, and I want another object to be able to react when something changes, and there's a pattern for that. The Observer pattern. So, these are just some specific patterns that we're going to talk about over the course of the course and the problems that they're trying to solve. And for this whole course, we're always going to start with what's the problem we're trying to solve and then here's the pattern that's going to fix that for us.

So What Is a Design Pattern Anyway?

So what makes a pattern? We've talked about the fact that patterns solve problems, but what are the — what's the makeup of a design pattern overall? Well, first of all it solves a problem. So we've started with that and that's great, but beyond that, it's a proven concept. Just because something solves a problem, doesn't make it a design pattern. It has to be a proven concept that's worked in a lot of different areas in a lot of different applications, and the solution of the problem can't just be obvious. Right. We don't have a semi-colon pattern, or we don't have a four-loop design pattern. Those are obvious solutions to our problem; these are a little more complicated problems that we're solving where the solution's not quite so obvious. It describes a relationship. Nine times out of ten — so I'm leaving an exception in there. But nine times out of ten, the design pattern itself deals with a relationship and how things interact in our code. It has a significant human component. This means we actually have to do something. There's some thought on our side that goes into this to make it really work as a pattern. So why bother with patterns at all? Why do we even care? So the first thing is why solve a problem again? If this problem has already been solved by somebody, why are we bothering to solve it again when we could just use their solution? A lazy programmer generally tends to be a good programmer, because we're not going to go make things complicated. We're going to take something that we know works and we're just going to implement it. It also gives us a common vocabulary and I think this is probably one of the more important reasons why we're going to do design patterns. Because this way, when we're talking to each other – and software development more and



more is a group collective thing. As we're talking to other developers about what we're doing, this gives us a common vocabulary to use so that we know what we're talking about easily.

The Types of Patterns

So let's take a minute and talk about the different types of design patterns that there are, and what we're going to look at over the course of this specific course. The first one is Creational design patterns, and these are patterns that deal with the creation of new instances of an object. If I want a new object, I'm going to use one of these patterns to create that for me and the first one is the Constructor pattern. And this is one you might be familiar with already if you're a .NET or a Java developer, and this something that you would use all the time. But in JavaScript it's a little bit different. So we're going to look at this a little bit differently and how it works on the JavaScript side. We're going to look at the Module pattern, and this is more if I just want a grouping of like methods or a service of some sort. We'll look at the Module pattern and how that works for that. And these two are more specific just to JavaScript. You don't see these talked about as patterns in other object-oriented languages. JavaScript has these two specifically, and we'll talk about why that is and what that looks like. The last two on the Creational we're going to talk about are the Factory pattern and the Singleton pattern. And Singleton's going to be a little different in JavaScript, because you'll find that actually a lot of things are Singletons that you might not have realized were Singletons. So it's a good idea to understand how the Singleton patterns work. The next type of pattern is the Structural pattern. Where Creational patterns deal with creating new objects, The Structural patterns deal with the makeup of the actual objects themselves. Now we're going to talk about three of these. We're going to talk about the Decorator pattern, the Façade pattern, and the Flyweight pattern. The last one is the Behavioral patterns. And these deal more with how objects relate to each other and how they operate, as opposed to the structure or getting a new instance of something. We're going to talk about three: We're going to talk about the Command pattern, the Mediator pattern, and the Observer pattern, and these patterns are ones that are fairly well used in JavaScript. There's a lot more patterns out there. If you read the Gang of Four book or some others, you'll see a lot more patterns than just this. But these are the patterns that are used most often on the JavaScript side, and I'm going to focus just kind of there so that you see only ones that you probably might use in JavaScript. Now here's how this course is going to work. It's going to be a little different than some other courses I've done, so here's how this is going to work. The first thing we're going to do is we're going to talk about objects in JavaScript. And that might be rudimentary for you if you've been doing JavaScript for a little while, so don't feel like you need to watch module 2 in this, because I'm just going to talk about how to create objects, what objects look like, and the three different ways to create them. And



we're going to talk about dot notation, and we'll talk about bracket notation, and object. create and all those things. But if you're familiar with all that stuff that I just said, you can probably skip module 2 and jump straight into module 3. If you're not familiar with that stuff, go ahead and take a look at module 2 to get your level set for when we start talking about the patterns. Now I'm going to work through these patterns, and for the most part the pattern is going to stand on its own. I'm not going to necessarily build one to the next. The first couple are going to set up a theme. We're going to build inside the context of a task object, but for the most part, each pattern's going to stand on its own, so if you only want to learn about the Façade pattern, that's fine, just go watch that series of clips on the Façade pattern. But for the most part, you can get away with either picking and choosing, or just watching the whole thing. We're going to describe each pattern. And remember that patterns solve a problem, so we're going to talk about the problems that we're trying to solve, and then we're going to solve that problem using the pattern. And so we'll talk about the rules of each pattern before we get going, then we're going to implement that pattern in Node. And if you don't do Node, that's okay, go ahead and watch the Node piece, because that's more of just the plain vanilla JavaScript version of that pattern. I'm going to execute it in Node, but if you're doing it inside the browser, it will work the same way. Then, we're going to implement it again in Angular, because Angular is kind of a different beast. And a lot of people are using it right now, so I wanted to take a minute to show you how that would work inside that Angular ecosystem, because it is going to be a little bit different. So that's the way this is going to work for this course. So let's go ahead and get started, and let's talk about some objects.

Objects in JavaScript

Introduction

Now before we jump too deeply into design patterns, let's take a few minutes to level set on objects in JavaScript. Now if you've been doing JavaScript for a while, this may be a little bit rudimentary for you. But it's important we take a few minutes and level set everybody to make sure we're all on the same page. If we're covering stuff that you already know, it's perfectly okay to just skip to the next module and dig right in on design patterns. But if you're not quite sure what the different ways to create objects are, or how inheritance in JavaScript works, or define properties or those types of things, hang out with us for a little bit, we'll walk through all of that stuff and get you ready to jump into the design patterns piece of this course.

Creating Objects in JavaScript



Now the first piece of this is creating objects, so we just want to create a new object in JavaScript. The simple way is just to use curly braces. `Var object equals curly brace, curly brace.` And a lot of times you won't see it this way; you'll put stuff in the middle of that, and we'll talk about that here in just a second. But this gives you just a clean, simple, one-off object that you can use to do whatever you want with. The next way is `Object.create`. `Object.create` is going to take whatever you pass in, and so in this case we're just passing `Object.prototype`, and it's going to wrap that in an object and hand you back the object. Now this, a lot of times, is going to be used when we start talking about inheritance. This is how we're going to do inheritance in JavaScript; we're going to create an object, then pass it in. The last way we're going to talk about is the `new Object`. Now the `new Object` has gotten quite a bit of slack, actually, and is a little out of favor, and I'm going to encourage you not to buy into the hype. Especially in ES6 we start to have the class operator that allows us to do true classes, so the `new object` really isn't as bad as some people are making it out to be. It's okay, and we're going to use it fairly extensively in this course in the right way, and doing the right thing. So, `new Object` just creates a new, empty object. This is the three ways that we're going to use to create new objects.

Reading and Writing Attributes

Now that we have objects, we need to assign some keys and some values to those objects. The first way is to use dot notation. So in this case, I create an empty object using the curly braces, and then I add `obj.param`. Using the dot notation I can add a parameter and assign that, in this case, new value. So I'm assigning a string to a parameter using dot notation. And then using dot notation again, I can get access to that. In this case, we're just going to log it out to the screen, `obj.param`, and that's going to write out new value. An alternative to this is using square bracket notation. So in this case, once again, we create the empty object, and then using bracket notation, I'm assigning new value to the parameter, `param`. Then I can get access to that parameter again using the same bracket notation. Now these two are interchangeable, so I can assign with bracket notation and I can pull the value out using dot notation, or vice versa, and either way. One of the advantages of using bracket notation, sometimes, not always, is that you can use bracket notation using variables. So in this case, I've got — I'm creating a new object, and then I'm creating a variable, just a string variable called `value`, and I can use that in bracket notation to get access to my new value. So in this case, I'm using the string `val` with `value` to assign new value into my object, and then I can use bracket notation to pull it back out again. Now, the advantage of this is if you're not sure exactly what's in that object, and you can use variables to store things and pull things back out.

Demo Task Creation



Alright. So for this course, we are going to be creating a task application. That seems to be the generic thing everybody wants to create, which is a task management application. So the first demo we're going to do is we're actually going to create a task object, using all three of the examples we just used, one-by-one, so we can kind of see the differences of them and how they work. So we're going to run this demo in Node, and if you don't have Node that's fine. You can download it and install it now. I'm using 4. 2; there's a 5. x version out right now I'm sure. They're all pretty much the same for the purposes of what we're doing. So in this case, we've got a task. js and we are going to create a task object. So var task = open/close curly braces. Now, a task should have a couple different parameters on it, so let's do task. title. We'll do task. description. And, let's do... Well, let's just leave it at that for right now. So, if I do a C. L., console. log on task. title. Okay, using task. js you can see we're spitting out My task. So I've got an object created, I'm adding things to that object, and I'm pulling things back out again. If I change this to my Object. create using Object. prototype, you'll see we get the same result. Those are the two basic ways to create this. Now what I can also do and put in here is a function. So I can do task. toString = function, and I can actually have that return this. title. Then I change this to toString, save that — you see; now I get my task. Now I could do my title, this. description, and now you see; I get My task and My Description. So here I've got an object. I've got two parameters, I've got a function all encapsulated, all in one thing. And the this in this case, refers back to my task Objects. So this always refers to the scope of where it's called, and in this case it's inside object, my task object, so that's why I get title and description. Now the last way we're going to just show you real quick is the new Object. Same thing. All the same works. See, they just all are interchangeable for right now. And in a lot of cases for what we're doing, you're just going to do curly braces. And I can actually add in here title and description right inside the curly braces. Title and description. Get rid of all that — if I run it, same thing. So that's creating an object and adding things to and pulling things from that object. Pretty straight forward. There's a good chance if you've been doing JavaScript for any length of time, you kind of know all this stuff already, but let's move over and do something a little bit different so that we can start to stretch our object creation just a little bit.

Define Property

Now sometimes we need a little bit more granular control over these properties that are sitting on our objects. And in ECMAScript 5 we got defineProperty, which is a method on Object that allows us to configure our properties to meet some criteria. So in this case, here we are defining property name on Object and then, that allows us to configure that property name. We can set the value to my name. We can set whether or not that's writable, which means I can create constants, so something that can't be changed. Enumerable means either in a for in loop, or if I do the keys function then it will show up or it won't show up, and



configurable means I'm not allowed to change the configuration of these items. So once I've set it, I'm not allowed to set it again to something different. So let's take a minute and look at that so we can see kind of what I mean.

Demo: Define Property

So let's take a look at define property and what the implications are of each of these configuration items that we have, and why we might use define property on our objects. Alright. So here we've got our task object that we created in the last demo. And basically, it's just a title and description, and then we've got a function to string that returns the title and then the description on it. Now if we convert this over to be Object. defineproperty, I can add this toString this way. So we're going to pass in task, and then the name of the property we're adding, and in this case we're adding toString. And then, the last parameter to this method is a JSON object. Let's clean this up. Alright. So, here's what we got. We got value, and in this case the value is just going to be the function, because we're adding that function to task. Now we add the three parameters. We've got writable; we'll make that true, we've got enumerable; also true, and we've got configurable, and we'll set all three of them to true for right now. Alright. Now, as of this moment, everything should just work exactly the same way and we can pop over here and do our Node and see, we're still getting our toString with the My Title and My Description. That's what we've got right here, this. title, this. description. Now, let's talk about why we might do some of these things. One thing that's very annoying about JavaScript is that everything is assignable. So if I say task. toString equals hi, I have now broken task, because toString is now equal to hi. So when I run this now everything breaks, because toString is no longer a function. ToString is now this thing, hi. So what writable lets us do is prevent that from happening. We set this particular property to false for writable, and that means that you can't accidentally or inadvertently overwrite this property later. So now if I run this, it's going to work. Now, in fairness, I did not get an error, nothing bubbled up, so be careful. But, I would rather not have my function overwritten than to have something weird happen. Alright. Now let's look at enumerable. If I just do a console. log on task, I see title, description, and then my toString function, and that doesn't look clean. Or, I can even do Object. keys on task and I get title, description, and toString. Now in this case, I don't want toString to show up in that way. I want — when you just a C. L. on my task, I just want to see title and description. I want to hide that toString from everybody else. So I can set enumerable to false, and now when I run this, now I just get title and description. So that's where those two things come in. If configurable is set to true, I can come back in later — this one is not as important as the other two. I can come back in later and redefine. So if I go, Object. defineProperty task, 'toString', I can set enumerable: true. And now when I run it, notice toString is there, and that's not behavior I necessarily want. I don't necessarily want somebody to be able to come in, if I set enumerable to false, I don't want



somebody to be able to come in and set it to true. So I keep configurable as false, and then I get an error. This actually throws an error, Cannot redefine property. It says hold on; you can't do that because configurable has been set to false.

Demo: Inheritance

And as we kind of talked about already, we can use the `Object.create` method for inheritance. We can, at least mimic inheritance for right now, using `Object.create`. So let's take a look at how we do that with the task object we already have created. So right now we have a task and our `defineProperty` with a `toString`. We'll get rid of that. Now what we're going to do is we're going to create an `urgentTask`, and that's going to inherit from task. So we'll do an `Object.create`, and we'll pass it in task. Now `urgentTask` now is essentially task; everything's the same. If we copy this though, and we change task – not an urgent task, we change this to is urgent and then we're going to `console.log(urgentTask)`. Alright, so let's run this and there you go, My Title is urgent. Now you know this is a new thing and we've actually created a new object, because I was able to change the `toString` on `urgentTask`. I wasn't able to change it on task, because notice, we set `writable` to false on the `toString` for task. But when we created `urgentTask`, now it's something new and I can change that.

Summary

Alright, now that we've done a quick level set on objects and how objects work in JavaScript, we can move on to the design patterns. But just a quick recap. We talked about three different ways to create objects. We had just curly braces; we had `Object.create`, and then the new key word. We talked about assigning and reading values using dot notation and the bracket notation. We talked about define properties, `Object.defineProperty` and the different options surrounding that. And then we talked really quick about inheritance and we'll talk more about all of these as we work through the rest of this course. Now let's actually dig into some creational design patterns, and we can talk about creating objects.

Creational Design Patterns

Introduction

So now it's time to talk about Creational Design Patterns. Up until now, we've talked about what design patterns are, where they came from, and we've talked about how objects work in JavaScript. Now let's actually take a look at some design patterns specifically, and how we implement them in JavaScript. Now creational design patterns are used to create new objects. So if I want a new



object of some type, I'm going to pick a design pattern that's used to create those new objects. I'm going to adapt how I create those objects to the specific situation involved at the time. So there's several design patterns involved, and we're going to talk about how and why we would use each one of them individually.

Constructor Pattern

Now we're going to start off with the Constructor Pattern, which, depending on where you look, isn't a design pattern from the original Gang of Four, but it's different enough in JavaScript that we're going to spend a little bit of time talking about constructing new objects. And so the Constructor Pattern for us for right now is used to create new objects with their own object scope. So if you're coming from .NET or Java or some other object-oriented language, this would be creating a new instance of a class. Now JavaScript doesn't have classes, at least not until ES6, and we'll talk about that here in just a little bit. But, JavaScript does enable you to create new objects out of existing objects, and so that's going to use the Constructor Pattern. So when I do the Constructor Pattern, we're going to use the `new` keyword. And the `new` keyword does a few things when you use it, when you stick it in front of a function. So, if you take a function, any function, and you drop the `new` keyword in front of it, it creates a constructor function out of that function. And when that's executed, it's going to do all four things; it's going to create a brand new object. So when I put `new` in front of a function, it's going to create a new object, which is kind of what we want to do. So that's the right thing. It's also going to link that object to an object prototype. If you're surfing the internet and you start looking at prototypes, it says that it inherits from, or it does things like that; it's not really the case. It links to that object prototype, and we'll talk specifically about that here in just a little bit. So we'll come back to that, but it links the object to that object's prototype. It binds this to the new object scope. So we'll talk about the `this` keyword and how that works, but it creates a new `this` and binds it to the object scope. And the last thing is it actually returns `this`. So if we look at some code, this is a constructor function in the Constructor Pattern. Now, like I said, any function can become a constructor function, but for what we're talking about, this is how we would lay out a constructor function. Now, what the Constructor Pattern allows us to do is to create a function and I can pass in some parameters to that function. And the name of the function is going to be the name of our object. So in this case, I'm just calling it `ObjectName`. I'm creating an object called `ObjectName`, and I'm passing in two parameters to my constructor. Because I'm going to use the `new` keyword, I have `this` now available to me and it's bound to this object's scope. So `this.param1` and `this.param2` are being assigned whatever I'm passing in. And I also have the option to do methods on `this`. So my object now has a `toString` method that's going to return `param1` and `param2`. This is the basic outline of how a Constructor Pattern works. Now, remember that the fourth thing that the `new`



keyword does is it would implicitly insert a return this at the end of this function. So, the very last line of this function would be return this and I don't have to do that explicitly, because the new keyword's going to do that for me.

Demo: Constructor Pattern

Alright. So let's try this out in our Node environment and see how this plays out in practice. So for us, we're going to do this inside the idea of a task application. So we're building an application that's basically just a task manager. So, I'm going to create a new file called task.js. And what I'm going to do is create a task object. So we're going to start by creating our task constructor. And if you remember, we create a task constructor just by creating a function. So in this case, I call it Task, and we're going to create a function that we're going to put everything in. Now in this case, I'm going to pass in name, so whenever I create a new copy of our Task object, I'm going to pass in the name and I'm going to create a task for that name in it. So, the first thing we'll do is this. name = name. The other item on Task that we want to keep track of — so if you think about a Task application where, whether or not it's completed. So in this case, the default is going to be false. Alright. Now I have a Task object, and it has this. name and this. completed. Those are the only two things. Now, what I want to do is I'm going to add a function to this. So this. completed = a function, and this function is going to change the value of completed. So let's just say that. So, this. completed = true. Now let's create one more. Let's do a save as well. So this. save. So once I make changes to a function, I want to save it to the database or something like that. And in this case, we'll just write out saving; we'll just say saving Task, and then give us the name. Alright, so now I have a constructor that has two properties, name and completed, and then two functions, completed — or, let's actually not call this completed; let's call this complete. And we'll put a CL on this too, so you see what's happening. We'll just say completing task, and then task. name. Alright. So now I have a constructor ready to go. Now to use that to create a new instance of this object, I am going to say var task1 = new Task and we'll pass in our task name. And we'll just say that's what we're doing. Now, I can create a couple of different instances of this, or to use the correct terminology; really, in this case, I'm just creating several copies of this object, because we don't have classes in what we're doing here, so I'm creating an object and then I'm just making copies of that object. So, there we go. And then we'll do constructors; we're going to do modules; we're going to do singletons, and we'll do prototypes. Alright. So, I have four copies of this object and they all have their own individual object scope. So, I can do task1. complete, execute that, and then we can save all of them. And you can see when I run this that they all have a separate this. So when I execute task4. save, up here the save, on line 11, you can see the this. name. And so, the this refers to the object scope of whatever object is calling the save functions. So, in this case, task4. save refers to the object scope of task4. So the this refers to task4. So what you'll see is this. name being, in this



case, create a demo for prototypes. Real quick, before we show this, let's make that this. Alright, now let's run this. There you go. Now what you see is there's completing task, save, save, save, for each of this different things. Alright. So that's the basics of the constructor. Let's talk about one other thing before we make this fleshed out a little bit more.

Prototypes

Alright now, one issue we have with what we're doing so far is that the function `toString` is reinitialized, it's recreated, for every task object that we create a copy of. So, if I have four task objects, like we did in our demo, this `toString` function was recreated all four times. And it's not really efficient behavior. That's not really how we want things to work. And so, what we've been given is this concept of Prototypes. And what a Prototypes is is basically just an encapsulation of properties that an object links to. So, I can have properties and methods that are on an object's prototype, and for every copy of that object that I have, it all links back to the same prototype. It's much more efficient to have one copy of all of those object methods out there, sitting inside the object prototype. So think about it like this: So, we have our task object, and that task object has a task prototype. So, in this case, the task prototype – we would put that `complete` and the `save` method on the prototype instead of inside the task object. So when I create two copies of `Task`, they both point back to that same task. Prototype. They don't copy, they don't inherit from, or they don't pull those methods into themselves; they are truly linked back to the prototype. So let's take a look at that and see how we would change things to work that way.

Demo: Prototypes

So let's take a minute and adapt our task constructor, our task object, into this prototype means of a constructor. So, basically what that means is we want to take the `complete` and the `save` out of the constructor and just attach it to the task prototype. And the way we're going to do that is by just using prototype. So its structure is `ClassName.prototype.methodName`, and then you pass in whatever arguments to the function. So in our case we didn't have any arguments, but that's the way that works. So here, we're going to go `Task`, our method name is going to be `complete`, and then, the function is just going to be this function right here. So I'll paste that down, get rid of our arguments because we don't need that, and now we have `complete` on `taskPrototype`. So now every time I create a new copy of `Task`, I'm not creating a copy of the `complete` function. That `complete` function exists in one place on the `taskPrototype`, and all of our task copies have access to that `complete` function, but they don't have their own copy of it. So let's do the same thing for `save`. So it's `Task.prototype.save`, copy our function, get rid of our arguments, there you



go. Now we have just the `this.name` and `this.complete` that get copied from one object to the next, and the `save` and `complete` methods do not. So let's run this again, and what you'll see is that nothing has changed. There you go. Everything still works exactly the same way, but now we're working in a way that is much more efficient.

Demo: Constructor Node

Now let's take a look at this in the Node environment and actually separate this out a little bit so you can see how to insert this Task object into a Node application. Now the first thing we're going to do is create our new file called `main.js`, and this will be where our actual application would be, so we'd stick everything over in `main`, and our `task.js` is simply going to be our Task object. So I'm going to copy all of that – or not copy it, but delete it, and move that over into `main.js`. So in `main.js`, we're going to actually create four tasks and then `task1` complete; all of this stuff over here. And our `task.js` is just going to be our Task object declaration, kind of the constructor declaration and how that works. Now in Node, it uses the `common.js` framework, so we're going to do `module.exports`, and we're actually going to export Task; that's all we're going to do. And what that gives us is a reference to this Task constructor. Now we're not exporting a newed-up version of it, we're just exporting Task. Now over in `main.js`, we are going to do a `var Task = require, and we'll do. /task`. Now that gives us `task`, and now, everything just works exactly the same way. So I'm newing up a `task`, `task1`, and everything's fine. So this is how we break that apart. On our `task.js`, we create our constructor, all the prototype items, and then do a `module.exports = task`. And then in `main.js` we're going to do exactly what we were doing before. We do a `require task` and then `new up our tasks` as we go right there, and I can call all the methods and everything just like that. If we run this, now, instead of running `task.js`, we'll run `main.js`, and everything still works. So this is how you start to break this out and use this in your Node environment. You create your objects in one file, and then you can use those objects by newing them up over in the rest of your application.

Demo: Classes

Now we'd be a little remiss talking about classes and constructors, and all of that stuff in JavaScript, if we didn't talk about EcmaScript2015, because in EcmaScript2015 we now have classes that we can use. So let me show you how to take what we just did with our Task object, and turn it into a class in EcmaScript2015. Now, we're going to do this on the Node side, and I'll show you how to lay that out because Node5 allows me to use classes, but only in strict mode. So you've got to do the `use strict` up at the top of your JavaScript file. And what that means, is that now this operating in strict mode in Node5 gives



me access to some of the EcmaScript2015 features. And so what I can do now is use the class operator on my object. And so if I take this task – I'm just going to take task and drop it in here into a new class.js. And what I am now allowed to do is I am allowed to use the class keyword. And I can say class Task =, and this is going to start to look a lot more familiar to everybody if you're coming from an object-oriented. NET or Java background, because now I'm going to create my constructor. I just have my class and now I have a constructor. And that function that I had before in my task.js, my task was actually a function. Now this function becomes my constructor. And so, now I have a constructor with this.name and this.completed. Now it's only half of it. I can get rid of my prototypes now, and take this function and actually just create my functions. So I've got complete and I've got save, and wrap that all inside of my class. So now, EcmaScript2015 allows me to just create a class with a constructor, and then methods and variables and all of that stuff can be on that task. If I take my main, I'll take these lines of code and drop them in class, just so we could execute this and so you can see what it looks like. We'll run this. Real quick you see we got an error and that's because we don't do that anymore; now we just have that. Alright. Now let me run it and you see completing task, saving Task, all that stuff still works. It's just a very different way to do pretty much exactly the same thing. And it might be a little more comfortable for some of you coming from a. NET or a Java background. Now you can do this right now using a transpiler if you're not doing Node. You can use Babel to transpile this down to EcmaScript5, and what that's going to end up looking like — If I use Babel to transpile this JavaScript file, it's going to come out looking something like this. It'll look a little bit differently, because they have reasonable code and they call some functions, but, ultimately, when you transpile a class that looks like this, it's going to end up coming back over and putting everything on the prototype the same way we do here. Alright, so that's EcmaScript2015 and classes, and you have access to that, either through a transpiler like Babel, or, if you're using Node5 or higher and you are running in strict mode.

Demo: Constructor Angular

Now let's take a look at this in the Angular environment and see how we would use this same Task object and the Constructor Pattern over on the Angular side. Now I have a very simple Plunker task management application built already; I went on Plunker. And if you go to the url you'll be able to see the same plunk that I'm using on your browser. Now I have a very simple task management application that basically just takes a list of tasks and dumps them out to the screen. It only has name and whether or not it's completed. If you go to script.js, you'll see I just have tasks and an array of tasks. And a lot of times, unfortunately, this is kind of the way we build Angular applications. You pull data from an API somewhere, it gives you back an array of items, and we just iterate over those items and dump them out to the screen. And then, at least show a



couple of problems with everything being in your controller. First of all, controller bloat means my controller are huge, I've got some services, but really, we're not doing much with those other than just pulling things out of an API, and it's not laid out in a very clean and maintainable way. And so, let's inject some objects into our Angular application so we can start to pull things apart a little bit and not have everything sitting in our controller. So the first thing we're going to do is we're going to take our Task object and we're going to put it into Angular. So we'll do a new file and we're going to call it task.js; same thing we did before. And like all Angular applications, we start out with an iffy so that we're not injecting anything into the global scope and we're going to look for our Task Manager application. Now in this, we're going to use the same Task; the thing that we used before. So we're going to create a function, and that function's going to take the name. And this Task is going to look exactly the same way it looked over on the Node side. Create a Task function has got a name, it's got completed, and then we've got our Task. prototype has complete, and save. This same thing. Now in order to put that into our Angular application, we need to add that to an app factory. So we're going to do app. factory, Task, what we're naming our object – so in this case we're naming it Task, and then a function and that's how that works. Now in this case, we're going to wrap our Task and our Task prototype calls inside this function, and then at the end of this function, we're going to return Task. So the same thing we did with module. exports over on the Node side, now we're just doing return Task on the Angular side. So basically what this does is this loads into Angular a Task object. So you're task constructor and your Task prototype, and all of that is now loaded into Angular and Angular says hey, I now have a Task that is a constructor function and some prototypes. And so now I can use that anywhere that I need to just by letting Angular know I need it. So if I come back over to script.js, I can now tell Angular hey, this taskController needs a Task. And now, I can new up tasks this way. Now I'm going to do something a little bit differently. Notice how right here on line 7, I have an object. I've got an array of objects instead of just names. So what – we're going to change our task a little bit to match this. I'm going to say new Task, and pass in the object instead of just the name. So we'll go back and we'll change our Task just a little bit, because I want to be able to pass in completed as well, instead of breaking it out into a bunch of different parameters, we're just going to pass in the object. So let's go back over to Task and change this up. Here we're going to just say data, and we'll do data. name and data. completed. Alright. Now back to our script.js. Now I'm using Task objects and my task actually has a complete function. So here, before when I was checking this checkbox, if we go to index.html, I've got an ng-click, I'm using my ctrl. complete method to complete this task. And I don't need to do that anymore, because my task, itself, if you look back in task.js; my Task has a completed method on itself. So now, in my index.html, instead of saying controller. complete(\$index), I can actually just go task. complete. And now let's run that and see what happens. Now I'm pulling everything back from inside my Task, instead of in my controller. So my script.js, I can actually get rid of this. complete completely and just have my tasks deal with all that functionality. If



I refresh, notice it stays the same. And just so we know, we'll set that one to false, refresh, and if we look at our console, when I click check we're getting completing task: task 2. If you remember, that's my save function on my prototype. So now, what I've done is instead of having all that functionality in my controller, I've moved all of the save and manipulate functionality out of the controller into my Task object. And my taskController actually shrunk by about five lines of code when I did that. Alright. So that's the Constructor Pattern for Angular and Node. Let's move on and look at a couple of others.

Module Pattern

Now the next pattern we're going to talk about is the Module Pattern. And really, the Module Pattern is used as a simple way just to encapsulate a group of like methods that I'm going to use on my application. So, think about it like a toolbox. It's just a simple way to take a bunch of methods that are similar, and I'm going to create a toolbox of functions for me to use. A lot of times, what you'll see with a Module Pattern is things like a service for my database calls or a service for my HTTP calls. Every time I want to do something with the database, I'm going to call this database module and I'm going to let that deal with all of my database calls. Now, a module pattern at its core is basically just an object literal. In this case, I'm creating a module as a variable that's just an object with a couple of methods on it, method and nextMethod. But they're key values, so in order to execute a method I'm going to do module.nextMethod and that's going to execute that function. Just a collection of keys and functions. Now once we wrap it in a function, we can start doing all kinds of fun things like create private variables. In this case, privateVar is not on the object literal that's returned in the function call, so therefore, nothing can get to it. It's a private variable that's sitting inside this module. Now the big difference between the Module Pattern and the Constructor Pattern that we were talking about before, is generally when I'm doing a Module Pattern, I'm only going to have one of something. So I've got a service that does all my database work. I only need one of those. And really all it is is a collection of functions. Unlike the Task that we were talking about before, on the Angular side we're creating a bunch of them, and newing things up. And that's great for constructors, but on the module side we don't new anything up. We just create a new instance of the module, just var Module =, and execute this function, and that's what we've got. So let's take a minute and let's look at how I do this in theory, and then we'll jump over to practice.

Demo: Module Pattern

Alright. Now let's take a look and see how we would lay this Module Pattern out in practice. Alright. So if you remember, we have a Task object and this Task has a name on it and it's complete, and we've got a couple of prototype methods, as



well, on top of this task. And then we have main.js that just creates four copies of this object, and then we save and complete a couple of times. What I want to do now is, I want to implement a database service that is a module that'll deal with all of our database stuff for us. Now to keep this simple, I'm not going to actually go through the motions of doing all the database work, because that's really outside the scope of what we're talking about here. But, I am going to put in the functions and methods so that you can see how that all would work. Now what we're going to do is we're going to create a new file called our taskRepository. Well let's rename that to be.js. Alright. Now this taskRepository is going to be responsible for doing all of our database calls. And so if you remember from the slides, basically, we just create a function, so `var repo = a function`, and then I'm going to return an object. And this object, this object is just going to be a series of methods that are going to be relative to database stuff, so database gets, database saves, those types of things; and then, I'll use these methods over in our main and our task in order to work. So, let's create a couple of methods. We'll do a get, and get is going to be a function that's going to take an id. And, we'll just write out to the console in this case, Getting task ' + id, and that'll simulate us getting something from the database. And then we'll actually return just a JSON object with a name, and we'll just call it new task from database. So basically, what this simulates is I've gone out to the database and I've pulled back a task and I've returned that task up to the calling function. Alright. So there's my api, essentially, for my repository. So now I'm just going to export `module.exports = repo`, and I can go ahead and execute this right here. So `module.exports` is actually going to be this return section right here. Alright. So let's try that and see if that works. So if I go back over to main where I have `var Task = require task`, now I can have `var Repo = require our taskRepository`. And now, I can do `task1`; instead of saying `task1` is this – let's just get rid of that. And now I can say `Repo.get` and pass in an id and wrap that in the new Task. So now I'm creating a new Task and I'm using my taskRepository to go out to the database and get that task back. Now one thing we want to change is, notice I was sending a JSON object back with name. In my Task objects, I'm actually passing in name. So let's just pass in a JSON object with some data in it. And you'll actually see this a lot. Instead of my constructor taking 12 parameters with all the different things, I'm just going to take a JSON object and have that object be my list. So there we go, `data.name`. Alright. So let's try this out and see if that works. What I should see is getting task1, and then, when I do `task1.complete` it should say completing task, and then the name of the task would be this one I pulled from the database, new task from db. So let's try that out. There you go. Getting task1, completing task: new task from database. So I have a method here in my repository that does my gets. Now that's only one; let's add another one that does saves. So comma save, and that's going to take a task, and in this case we're just going to do a `console.log, Saving task.name to the database`. Alright. So now, whenever I call save on my repository, it's going to save it to the database. Now instead of doing this from main, let's do this one from task. So I'm going to do the same thing, `Repo = require('./`



taskRepository. Now here, when I'm doing Task. prototype. save, console. log – instead of writing that, well actually, let's leave that there, and now we're going to do Repo. save, and I'm going to pass in this. Now in this case, this is the Task. So I'm passing in this. name and this. completed into my repository. So now when it says Saving task. name, we should see that. Now, if we go back to our main call stack, task2. save, 3. save, and 4. save, we should see demo for modules, demo for singletons, demos for prototypes, all being saved to the database. So let's run that, there you go. To the database, to the database, and now everything's undefined. And the reason for that is because when we do our new task here, we didn't change this to be an object. So let's do that. Alright, now that I've changed that, there you go. Now when I run it, you can see saving, create a demo for modules to the database; saving, create a demo for singletons to the database. This is a module; this is how this works. So basically, I've taken all my database code and I've encapsulated it in this repository object and gone from there. Now the reason why we want to do this is when this function is executed, I can actually create a variable and instantiate my database connection, and do all my database work here. And because of the way JavaScript closures work, every method that gets called – so my get method and my save method, has access to that database stuff up above. Nothing else has access to it; none of my calling functions can get to it. My database is secured up in this module, but all of my methods, my get method and my save method, can have access back up to that database. Now one slightly alternate method, and a method I actually prefer for this, is called the Revealing Module Pattern. And it basically means instead of doing this, I'm going to get rid of that and I'm going to create my function up here, and I'm going to define my methods up above. And then down here in my return, I have my list of my methods. And this way I can just go to my return statement, anywhere I can go into a module; I can go to the return statement and I can see what methods are available on this module. So it's just a basic little documentation for my API that says look, this module has a get method and a save method. And if I want more details, I can go up and look at them up above. So that's the Module Pattern in practice in our Node environment.

Demo: Angular Module Pattern

Now let's take a look at this on the Angular side, and build that same repository using the Module Pattern in our Angular application. So, so far we've got a task manager application that has a task. And basically, this task has complete and save; it's got two functions on it. And it's doing this save, and basically all we're doing is console. log('saving task:', but we don't want to mix our database code, actually on the Angular side our API code, so this would be calling back to an API; we don't want to mix that API code into our Task. We want something else to deal with that for us. So what we're going to do is we're going to create a TaskRepository. And that TaskRepository is what's going to deal with all of the API calls for us. And so then, down here, what we want to be able to do is do a



TaskRepository. save and pass in this; pass in this task object and have that deal with all of that database stuff for us. Now, the first thing we're going to do when we do that is we're going to create a new file called taskRepo. And this is where we're going to build that module. If you remember, in our other code, in our Node code, we had a repo, get and save, and that's all it had on it. We're actually going to copy this function and then we're going to use that over here. So we're going to start with our iffy and we're going to get our Angular application; that's the first thing we always want to do. So var app = angular. module, and we want taskManager. That's what we created it over here as, angular. module taskManager, so that's what we're going to pull in. Alright. So we're going to create a taskRepo and that's going to be a function. And we can actually paste this same code; get rid of the double function there, and ask for \$http and it works the same way. So I now have a task repository with a get function and a save function, and then we're using that Revealing Module Pattern where we have a get and a save. Now the only thing we still have to do – let's beautify that a little bit so we know what's going on, is add that to our Angular application, so app. service, and we are looking for TaskRepository, and we're going to pass in this taskRepo function that we've created. So basically, all this is – it's the same thing we did on the Node side. This is an object with a get and a save, and we're returning get and save; it's going to handle all of that code for us. And then over on the task side, we're just going to call that. When we click Save, we're just going to call that save to this. Now one other thing we want to do. Notice we're never actually calling save. So, Task. prototype. complete, when we complete a task, when we set to completed we're also going to save it. So let's do this, this. save. Alright. So what we should see in our console when we run this, is we should see completing task, task1; we should see in our repository Saving task1 to the database when we do all that. So I'm going to click — ah, one very important thing we did not do is add our taskRepo, there we go, to our scripts. So okay. Now I'm going to refresh, and when I click task1, completing task: task 1, Saving task 1 to the database. So now, all of our HTTP code and all of our API calls and everything like that can be pushed out to this repository built using the Module Pattern. So we just have a module that handles all that stuff for us. Now, on the get side we could do the same thing. We're not going to do it right now, but I would challenge you to go ahead and try it. If you go to your script. js, here I'm pulling in some tasks. This would be the same thing. I would pull these in from my taskRepo and then create new tasks out of it.

Factory Pattern

So the next pattern we're going to talk about is the Factory Pattern. And this is a pattern that is used to create objects for us, and sometimes the creation of an object can be complicated, either because of configuration or other factors, and a Factory Pattern is used to simplify that and hide a lot of that from our frontend. So basically, a Factory Pattern comes down to a couple of things. 1)



Simplifies object creation like we just talked about. If I have a repository, I've got to open a connection to the database, and I've got to you know, set up caching and all of those different things that I don't want to do in my controller, or in my routes. And so, I'm going to use a factory to deal with all of that code for me. It's also good at creating different objects based on need. So if I need different types of objects, based upon where I'm at, that's something that the Factory Pattern is good at. And we're actually going to look at that in our example, because we're going to do repository creation. Now we've already created a task repository and we just call out to the task repository to do everything we need. Now on my controller though, I might have a task repository, I might have a user repository, I might have a project repository if we're going to split up our tasks by project, and I don't want to have to pull all of those different repositories into my route, or my controller. So what we can do is we can create a repository factory that's going to pull in the necessary repository that I need at the time, and our controller doesn't have to worry about all of that.

Demo: Factory Pattern

So let's take a look at how this works on the Node side when we're dealing with factories and repositories. Alright. So what we've got here is an application that's going to pull in three different repositories. We've got a task repository, we've got a user repository, we've got a project repository; and what this application is doing is then using those three repositories to new up a new task, and add a user to the task, and add a project to the task, and then we're going to save that out to the database. So what this ends up looking like is kind of junky, because up here at the top you've got three different repositories that you're pulling in, and it can kind of add a lot of stuff to this main.js and that's not what we want to have happen. But what I'll show you real quick is what this looks like when we run it just so you can see. Notice we've got – we're newing up a task repository, getting task 1, user 1, project 1; then we — there's our console.log task, there's our task, and then we're saving it to the database. Okay, what we're going to do is we're going to add a factory, and that factory is going to replace lines 2-4 and get rid of those so that we're not having to change main.js or add repositories in here every time we want to pull from a different data source or something like that. We want something else to deal with those things for us, and so we're going to use the Factory Pattern to create these repositories for us, as necessary. So let's look at a repoFactory. Now, this is just a simple factory with a function called getRepo. And that function takes a string that's just the repository type, and then checks what type of repository I'm looking for: task, user, project, and will then return a require to that task repository, or user repository, or project repository. So the very simplistic thing that this looks like is, hey, give me a repository and I'm going to give you the type. So in main2.js we've converted this, and now I can call repoFactory.getRepo and the task, and that's going to return back an instance of task repository. And then, it will give me a user or a project based upon what I'm



sending back. Notice up on line 2 I only have my repoFactory. Over in main.js, the way we were doing this before, I had three repos up here. Now, I just have the Factory. Now this isn't quite super awesome, because I've added something to my task repository that down here on line 16, console.log, newing up a task repo. And when I run main2.js, I'm newing up task repos all over the place. So this isn't the end product, this is just where we're starting. So what we're going to do instead is we're going to cache our repository, so I've created a new one, repoFactorywCache. And now if you look, repo equals task; then, I check to see if this.taskRepo exists already. And if it does, I just hand it back. Otherwise, I new up a new task repository and return that. What that does for us is, instead of creating a new task repository every time; I can just go and cache that in my factory. One thing to keep in mind is I might have config code right here. As part of the benefit of the Factory pattern is I can drop all my config code in the factory before I return it, and then I don't have to deal with that over on my main.js code. Alright. So let's look at our main2.js where we're using the repoFactory, and now let's switch it over to use the one with cache. Now what you'll see when we run this, if you look here, we've got three newing up task repos. This first one and the third one are from creating these two new tasks. The second one actually comes from our task where it news up its own task repository, and I'm going to leave that there for right now. So what you're actually going to see is we'll just drop one of these; we'll drop the second one and this will be coming out of the cache. And we can actually say, come right down here and put a console.log in, Retrieving from cache. Let's run that. Alright. See now that newing up task repo here became Retrieving from cache. And now we're caching our repository as kind of just as they're needed. We're not loading them all up right away; we're just caching them, as they're needed, and then running them that way. Alright. So that's a simple factory. Just, hey, I'm going to go and get something and pull it back. If you look at our code though, this repoFactory.getRepo, task,.get, that kind of clued you in; it feels kind of icky. So there is one other way we're going to look at to use our factory to clean this look up a little bit. So the way we're about to implement the Factory Pattern actually is used in gulp-load-plugins. So if you use gulp at all, gulp-load-plugins is a fantastic tool that basically runs this Factory Pattern over all of the gulp plugins that you might use for your code. And let's take a quick look at what this looks like. So basically, you do var gulp = require(gulp), and then gulpLoadPlugins = gulp-load-plugins. And what gulp-load-plugins is going to do is it does all the requires for every gulp- thing. It looks in your package.json file and all the gulp- stuff, it just loads that on its own. So now I have plugins.jshint, or plugins.concat, and that just simplifies everything up in your — the top area of your gulp file. We're going to take this same basic idea and do it for our repositories. So what I want to do, if you look at main3.js now; I want to do repoFactory.task, or repoFactory.user, or repoFactory.project. Main2 we had to do repoFactory.getRepo task and.get and I don't, like I said before, that looks kind of cludgy. So what we're going to do now is we're going to implement a factory that lets me do this: RepoFactory.task. Now you can actually change repoFactory to just repos and have it look nice and clean. So let's look at how we



implement that. So the way we start here is I'm going to create a copy of this, called repos. So repos just refers back to the object scope for repoFactory, because down at the bottom when we do a module. exports, I'm newing up that repoFactory. So, this refers — or repos refers to this object scope for me repoFactory, and I'm going to use that here in my forEach callback. Now, I have a list of repositories, and this list could come from anywhere. I could look at a specific directory. I could just go out, hey, let's go out to my /repo directory and load all of the files that are in that repository directory and drop everything before repository. So now I've got taskRepository, the name is task, the source is. /taskRepository. Then I just do a simple repoList. forEach and repos. name; remember, when we were talking about objects, I'm using bracket notation now, because I'm using a variable to add. So repos, bracket notation repo. name, so. task,. user,. project, = require, and I just require my source. So, repos, bracket task, equals require equals. /taskRepository. Repos user equals require, and I just run all those requires right there. What that does for me is now, look at back at main3. js, I have this very simple, clean, repoFactory. task, repoFactory. user. And the factory is taking care of creating all my repositories for me. I don't have to worry about any of that stuff, and pulling all that back, and now, in my main it's just the simple little, hey, I've got a repoFactory and here I'm pulling tasks, I'm pulling users, I'm pulling projects and everything just works. And if we run main3. js, there we go. Everything just works. Now, notice we're not pulling anything from cache, and that is because everything was cached inside my repoFactory. So when I created a new repoFactory, it just cached it in my repoFactory automatically, and then it dealt with all of that caching for me.

Singleton Pattern

The next pattern we're going to talk about is the Singleton Pattern. And the Singleton Pattern is used to restrict an object to just a single instance of that object across your entire application. Now the way a singleton works is that it remembers the last time you used it, so the singleton as an entity remembers the last time you used it, and then hands you back that same instance that you used before. So typically, a singleton looks something like this. In this case, sticking with our repository idea that we had from our factories, we've got, in this case, a task repository that we're going to create as a singleton. Now the first thing you're going to do is create a taskRepository object. And in this case, we have a var TaskRepo, and that taskRepository is this object that we're going to create. And we have a createRepo function that creates a new instance of that repo when we're ready for it, and then it returns that back. So when I want a new instance of my repository, I call this function, getInstance, on my singleton. On my taskRepository singleton, we have a get instance function that checks to see if not a task repository; if that taskRepository hasn't been created yet, I'm going to create one. Otherwise, I'm just going to return the one that I've already created. So basically what happens, when I create a new repo1 and repo2, I'm



calling `getInstance`. And when I call `getInstance` for `taskRepository`, it just calls out – the first one creates a new instance, and then the second one already has that instance, so it just returns it. And actually, if you check with an `if`, they'll be the same `taskRepository`. Now one thing to remember when we're dealing with this: `Node.js` uses the `CommonJS` module pattern, and `Angular` even does something a little different that kind of makes the creation of singleton in our applications a little less painful. All of this stuff we just looked at about `getInstance` and all those things, we don't have to do in `Node` because of the way it uses `CommonJS`. Or, we don't have to do in `Angular`, because of the way it creates its services. So let's look at how we create singletons and how singletons work in `Node` and `Angular`, so that you understand kind of how that `CommonJS` pattern works so you don't get stuck using the singleton if you don't want to be, or you are using a singleton if that's the idea that you want to do.

Demo: Singleton Node

So let's look at `Node` in this `CommonJS` module pattern, and how singletons work in that environment. Now I've created a repository, just a simple repository, that has one method called `save`. And I've added a, just an integer, number, to this repository called `call` that's going to keep track of how many times that `save` method is called. And then when you call `save`, I'm just going to write out to the console saving whatever we've written and `Called` has been called, however many times. Now we have a `main.js`, and this `main.js` is creating an instance of `taskHandler`. So `taskHandler`, if we look over here, creates an instance of the repository, and then `taskHandler.save` is just going to write out `myRepo.save`, it's going to create a new task in the repo, `Hi` from `taskHandler`. So every time I call `save`, it's just going to call `myRepo.save` again. And then `main`'s going to do the same thing. So right now if I run this, `myRepo.save`, I'm going to have three from `main`, and then I should have four from `task handler`. So when I run this, notice I get two newing up `task repos`. So I've created two instances of this `task repo`, and then I save from `Main`, and I `Called` it 1, 2, 3, and then saved from `taskHandler`, `Hi` from `taskHandler`, called 4 times. So we obviously don't have a singleton of my repository, because I've got two different instances of this repository. One instance is sitting in `main`, this instance right here, and then `task handler` has an instance of `myrepo` as well, but it has its own instance. Now, what `CommonJS` does, if we look at the `Node` documentation, is caches everything that is in a `require`. So if we look at `Caching`, right here in the first paragraph, `Modules` are cached after the first time they are loaded. Every call to `require` of something, I'm going to get the exact same object return. So this third paragraph is what we're doing right now. If you want to have the module executed code multiple times, export a function, and call that function. So what I'm doing right now in my repo, `module.exports` in my repo, `module.exports = repo`. I'm just exporting this function, and then I'm requiring `main` to execute that function in order to get a repository they can use. A lot of times what you'll see is a `require` and then you



just stick that at the end of it to execute it. So when I do that, what's being cached in the module export is the whole function. Now what I can do, and what a lot of people do, is I can execute that function in my module. exports. And when I execute that in my module. exports, I have effectively created a singleton of my repository. Because what Node.js is going to do is it's going to store that instance of this return object, this object right here that's returned, it's going to store this and the closure around this function means called and save, and all these things are going to stay the same. So now, here, I don't have to execute it any more, so I can get rid of a couple of things. Let me change this around, change that to myrepo, get rid of that line, and over on taskHandler I'm going to do the same thing. I'm going to get rid of myrepo, and change that there. Now, the only thing I've changed is I'm not executing the function, because I'm executing it over on myRepo when I do my module. exports. Now what that means is now when I run this, I have effectively created a singleton of my repository, so I'm only going to see one newing up task repo, and I'm going to see 1, 2, 3, 4, 5, 6, 7, 8. So when I run it now, I only have one newing up task repo, 1, 2, 3, 4, 5, 6, 7, on my execution. So that's a very clean, very simple way to create singletons on my Node side, and you see this happen a lot. Especially if you're doing Mongoose or MongoDB or Express. Module. exports usually will export either a repo like that, or I could actually new it up and do new repo and it's going to work the same way. And you'll see this a lot as you look at packages. A lot of packages will just execute a new instance of whatever it is they're doing, and they have effectively, because of the way Node handles their modules, they have effectively created a singleton of that object.

Demo: Singleton Angular

Now let's look at Singletons in Angular, and this is actually going to be pretty easy and straightforward, because in Angular all services are singleton. That's just the way it works. Actually, if you come over to that Plunk that we've been using, I've changed some things a little bit, but if you look at our task repository – get rid of this for right now. At the bottom where I do an app. service, taskRepo, what app. service does is it actually calls new taskRepo. It goes ahead and news it up right there, and then serves that back and forth. And because it does that, because it news it up automatically, just the same way we were dealing with the Node side, it's going to hand that same service back and forth to everything. Now let me demonstrate to you what that looks like, just so you trust me that you know that this is what it does. So I have the taskRepo, but I've added this called variable in the same way we had it over on the Node side when we were looking at the Node singletons. And I have a get method, and I increment called and I added called on the end, and I have a save method doing the same thing. Now I'm using this task repository in two places. The first place is on script.js just in my main controller. When I'm building out my task lists, I'm pulling from TaskRepository 1 and 2. And so what you will see is, when I run this, if I pull up my



console, you see right here down at the bottom, Getting task 1 called 1 times, Getting task 2 called 2 times. Now I'm also using this repository over on our task. When I do a save, TaskRepository. save(this), what save is doing is just writing out to the database, or in this case, just the console, saving task. name to the database and I've added that called. So, task. js is using TaskRepository just the same way that my Controller is using TaskRepository. Now in task. js, execute save, what you see happen is Saving task 2 to the database, have now been called 3 times. Saving task 1 to the database, have now been called 4 times. And every time I click this, I'm just incrementing. So, singletons in the context of what we're talking about are fairly predominant; almost everything's a singleton. If I new it up on the Node side before I hand it back to my module. exports, then that becomes a singleton just by virtue of the way CommonJS works and the way modules work in Node. If I create a service in Angular, then that service becomes a singleton. That's just the way Angular's going to work when it's dealing with services. So that's the different way we create objects. Let's sum that up real quick.

Summary

So to sum up, we've been talking about Creational Design Patterns. These are patterns that we use to create new instances of objects whenever we want something done. And we talked about a few. The first one being the Constructor Pattern. And the Constructor Pattern is what we use to create a new instance of an object. And typically with the Constructor Pattern, that's when we're going to have a few of something. So in our examples we used tasks; I wanted to create multiple tasks. And we talked about prototypes and how the prototype worked on JavaScript. We talked about the Module Pattern. The Module Pattern is when I want to bundle a group of like things together and create just a bundle of methods. We used repositories as just a bundle of methods that we could use to interact with the database or an API or something like that. We talked about Factories. We used repositories for our factories and we created a factory repository that would create these repositories for us, so that we don't have to have all those repositories listed out in our requires on our main JavaScript. And the last is Singletons, and we talked about how Singletons work in the modern era for JavaScript, in Angular and Node, just so that we know that a lot of times we're dealing with Singletons when we don't really realize we're doing it, and so how to avoid that situation if we want to on the Node side, and understand the implications of dealing with Singletons. Now that's Creational Design Patterns, now let's take a look at some other types of design patterns in the next module.

Structural Design Patterns



Introduction

So now we're going to start talking about Structural Design Patterns. Now structural design patterns are concerned with how objects are made up, or what the composition of objects are, and how to simplify relationships between objects. Now, if you remember back in module 1, I said that design patterns are all about relationships. And in Creational Design Patterns we didn't see a lot of relationships, because we're interested in creating an instance of an object. The only one that dealt with relationships, really, was the Factory Pattern, where it was creating other objects. Well in structural patterns it's all relationships. It's all going to be about, let's take one object and how this new object relates to something else. Now Structural Patterns, like I said, deal with the relationship of an object. And they do it in two ways. And the ones we're going to look at will either extend the functionality or simplify functionality. So I've got two extremes here. We're going to talk about the Decorator Pattern, which is going to extend the functionality of an object. We're also going to talk about the Flyweight Pattern and the Façade Pattern, which are going to help simplify functionality, or at least mimic simplifying functionality, to make things easier to use. So it's all going to be about how one object relates to a separate object.

Decorator Pattern

So the first pattern we're going to look at is the Decorator Pattern, and this pattern's used to add new functionality to an existing object without being obtrusive. And there's a couple different levels to this that we're going to look at. We're going to look at simple, just adding new features to an existing object, and then we'll look at true sub-classing and how that works in JavaScript. Now, one of the things we're going to talk about here is more complete inheritance. How I can actually fully inherit an object in JavaScript, which is a little bit different than how you would do it in .NET or Java. We're not going to only do that, but that's one of the things we're going to look at. The Decorator Pattern wraps an object, so you could take an object like Task, in our case, is what we've been working with. Object already works, and so I want to protect the existing object because it's there and it's used everywhere in our application. I want to add some functionality, or I want to wrap that functionality with some additional features that weren't there before, without having to change Task itself. And so what we're going to look at is we're going to look at a couple of different ways to add what we would call an Urgent Task. So something with a notify method or a deadline, or something like that, so that we can create a new decorated Task that's not going to change the way Task works itself.

Demo: Simple Decorator



Alright. So let's take a look at just some simple decoration to see some options that are available to us without a whole lot of effort. So here we've got a Task constructor, and basically all it does is create a name and completed. This is the same Task we've been using kind of all along, if you're following along. It's got two methods, complete and save, and those things do things right now. Now what you'll see is that we're creating a task and we're completing it and saving it. So when I run this, you're going to see, I'll just run it again, completing task, my Legacy Task; and saving Task, my Legacy Task. Now the problem that the Decorator Pattern is trying to solve is I don't want to mess with Task, because myTask is working. And I don't want to take a chance of implementing some bugs into myTask when I'm trying to create something new. And so what we're going to create, and I'll fold some of this stuff up, is we're going to create an Urgent Task. And so we'll do `var urgentTask = a new task`. And we'll just do the same thing on all of this. Instead of Legacy Task, now we're going to be Urgent Task. So now I've got an Urgent Task and a Legacy Task. But what I can do, and one of the cool things about JavaScript, you don't get to do this other places, is I can just start changing stuff about this object. So I can say `urgentTask.priority = 2`, and I can even create a new method on this. So I can do `urgentTask.notify` and make that a function and say `notify` is going to just write out to the console right now, but I could say, hey, I'm notifying important people. And so then my `urgentTask` can now `notify`. And so if we run our code, you'll see, I'm completing my Legacy Task, but now I'm notifying important people and completing my Urgent Task stuff as well. So I have decorated `urgentTask` with a priority and a new function called `notify`. And that's great, but I can even take it a step further and I can add `notify` to my `save`. So I can say `urgentTask.save = a function`, and I can do this. `notify` and then I can do `Task.prototype.save`, and we can use `.call`. And what `.call` is going to allow me to do is use the `(this)` keyword and bind this `save` execution to the `(this)` scope. So now, I've overridden `urgentTask` with `save`, so that now I'm going to `notify`, and I'll get rid of this `notify` so that we can see it working, and then I'm executing the `save` as well. And so now I'm going to do `urgentTask.save`. And what you'll see when I run this, is now we're saving Task, Urgent Task, and there's our notifying important people. So, when I do `urgentTask.save`, I have decorated `urgentTask` with a new `save`, and I'm notifying important people, but then, I am also calling the `save`, the original `save`, off the `Task.prototype`. So this is simple decoration. I'm taking a task and I'm decorating it with some additional features and functionality so that I don't break the existing task object.

Demo: More Complicated Decorator

Now let's do something a little bit more complicated, because I created Urgent Task and that was great, but if I want more than one Urgent Task, I don't want to have to do all that work again. And so if you look at the way our Task was, every time I create a new Urgent Task I have to go through and add the priority and add



the notify again, and that's painful. So what we really want is more true sub-classing, or since JavaScript doesn't have subclasses, we're going to call it sub-objecting, even though that's not really a word; we're going to use it for right now, because I want to create a new object that rocks that Task and allows me to decorate it with my notify and my priority and all that, without changing Task. So let's pop over to Task, actually, we'll just copy all this over to a new file. So I've got myTask back, but now, what we're going to do is we are going to create a subclass, or a sub-object, no classes. We're going to create a sub-object that's going to wrap Task in its own thing so I can create more than one of them. And the way we're going to do that is by creating a new constructor. We're going to do `var UrgentTask =`, and we're going to have a name, but also pass in that priority. So, if you remember we wanted to add priority, so we're going to add that. And in. NET anyway, you would call super to execute the super class; in this case, we're going to use that call keyword again, and we're going to call the constructor of Task to go ahead and execute everything there. And so if you remember Task, would say `this.name = name`, `this.complete = false`; so when I do a Task. call, I'm going to pass in this, and then said name. And then I'm going to do `this.priority = priority`. Now, if I create a new UrgentTask with, let's say 'This is urgent' and a priority of 1, when I print that out, UrgentTask should have three things on it. It should have a name and completed from Task, but then, it should also have priority from our UrgentTask constructor. So now, if we do our sub-objecting, you'll notice our Urgent Task has a name, 'This is urgent', completed is false, and a priority is 1. So I have a true sub-object from Task. Now I don't have everything and that's a problem, because I don't have the prototype, and I can't do `ut.complete` like I can with `myTask.complete`, because I don't have that prototype, so I have to add that. And so the way we do that is we use `object.create`. So I can do `UrgentTask = come up here. UrgentTask.prototype;` I could just set it equal to `task.prototype`, but the reason why I don't want to do that is because that just assigns them to each other. So if I start changing `UrgentTask.prototype`, it's going to start changing `Task.prototype` too and I don't want to do that. So what we're going to do is we're going to use `Object.create`, and we're going to pass in our `Task.prototype`. And what that's going to do is make a new object for my prototype out of `Task.prototype`. So those are different, and I'll show you that they're different here in just a little bit. But for right now, let's see if I can complete my UrgentTask. So now when I run my sub-object, I'm seeing completing task: This is urgent. So now I have a true sub-object. I've got a true inheritance going on, where I have my constructor and I have my prototype all coming down to this new object, UrgentTask. But now I want to change some things. Remember over on the task before, we were changing `save` to do something. And we're going to do that again in this sub-object, because now I can do `UrgentTask.prototype.notify` and have that equal the same function. So I can just copy everything over from before, but now it's actually on the prototype of UrgentTask and I can also do the `save`. I can do `UrgentTask.prototype.save` and we can do some special stuff before saving, and then, do that same `Task.prototype.save` and we use the call like we did over here



on task where we added save. call this, I can do the same thing here. Task. prototype. save. call with this, and now this refers back to UrgentTask, or whatever that object scope is that's there. So now, when I do a ut. save – let's add notify in here too. So we can do this. notify; I can do this. notify, do some extra special stuff, and then save it all through here. So let's run that. Now look what we've got. We've still got our Legacy Task stuff; nothing's changed on my Task. Legacy Task is still working, and that's the beauty of the Decorator Pattern is because I don't want to mess with Task, and I have zero risk that Task is going to break, but now I have completing task: This is urgent, I'm notifying people, I'm doing some more special stuff before saving, and then, saving the Task exactly the same way that Task is saving. I'm calling this same method, Task. prototype. save. So I haven't changed anything, I've just wrapped it with some additional stuff. So this is more true sub-objecting and true inheritance that we have going on inside this more extensive Decorator Pattern, but you've got two options here. We've got very simple decorators in the Task.js where we're just adding a couple of properties and methods onto a new instance of Task. Or, we're truly inheriting and creating a whole new thing called UrgentTask that I can create multiple versions of. So that's the Decorator Pattern in a couple of different forms. Let's see how decorators work over on the Angular side.

Demo: Decorating Objects in Angular

Now let's start looking at how to do these same decorating things in Angular. And there's two different ways we're going to look at this. We're going to look at decorating object in Angular, which will be fairly similar to what we just did in Node, but we're also going to look at decorating services in Angular using the decorator method that Angular provides for us. So first and foremost, let's do this Urgent Task over on the Angular side. Now here we are in our Plunk, same Plunk we've been using all along, and I've got my Task Manager, task 1, task 2, and if I open up Developer Tools and open up the Console, you can see we're writing some stuff out to the console every time we do something; so completing task1, completing task2. Now what I want to do, we've got our task, is I want to create UrgentTasks. I want to do the same thing. I want to be able to say UrgentTask, leave these first two as tasks, but then create UrgentTasks as well. So I want to be able to have be able to have task 3 and task 4 be UrgentTasks, while task 1 and task 2 are Tasks. But the same save, the same complete, and same save functions as completing task and the save to the repository need to stay intact, because I don't want to break anything that exists. So what we're going to do is we're going to create a new file called urgentTask. And when we do anything in Angular, we're always going to do our iffys. So I'm going to do function, and then all of our curly braces and parenthesis, and now we're going to actually create this urgentTask. Alright. So we're going to start by pulling in app, Angular. module, 'taskManager', and then we're going to do app. factory. And actually, if I pop over to Task, the way we do this, this app. factory('Task', and all of



that; let's just copy that so we're not typing everything over again. Now that's not ideally what we want, right? So we want to not have to replace everything, so we're going to do UrgentTask, and our UrgentTask constructor is going to be the same as what it was, actually, over on the Node side. So let's actually do that. From this point, I'm just going to copy everything. So we've got Task. call, our Task. call adding things to prototypes, the this. notify, the task. prototype. save. All of that stuff is going to work just fine. Let's just do this. Paste that there, beautify it all; we'll minimize this. And then we're going to return UrgentTask. So notice, I haven't changed the complete function, the complete function from Task is still going to be the same, so completing task. When you see that, that's coming out of Task. I'm still calling Task save, which really just calls our repo, but now, when I run this, I'm going to have two UrgentTasks and two regular Tasks. One thing we still have to do though is add urgentTask to our index. html. Spelled correctly. There we go. Now we missed one thing, so let's look at what's in our developer tools here real quick. Task is not defined. That's right. So an UrgentTask, we're pulling in TaskRepository; we also have to remember to pull in Task. So let's refresh that. And the last thing. When we copied it over, there's a couple of things we've got to do. In Angular we were data, so let's fix that – data. priority. And then we're passing data into our call. We just did it a little differently on the Angular side that we did on the Node side. So now we'll refresh; there you go. So if I clear everything out, when I click task 1, task 1 is just a regular Task, so I'm going to get completing task: task 1 and then Saving task 1 to the database. When I click on task 2, those are both just regular Tasks. So, completing task and Saving task. When I click on task 3, task 3 is an Urgent Task, which means that when I complete task 3, nothing's going to happen, nothing's going to be different. It's just going to work the same way as Task. If you look in UrgentTask, notice there's no complete here. So when I click on Complete for task 3, it's calling Tasks complete that it pulled out of task. prototype. But, when I click on task 3, the save is going to say 'notifying important people', doing special stuff, and then saving task 3 to the database, called 7 times. So there you go, completing task: task 3 that came out of Task. Notifying important people, doing special stuff, saving task 3, all of that stuff came out of the Urgent Task. So for this piece anyway, there's not a whole lot different between Angular and what we did on the Node side. All we did was create a new UrgentTask and pulled in Task and wrapped it up with all this other stuff. So the decorator for an object works pretty much the same on the Angular side as it does on the Node side. One thing we're going to do in addition to this is we're going to decorate a service and see how that works differently using Angular's decorator method.

Demo: Decorating Services in Angular

So now, let's look at how to decorate services in Angular by taking a look at our taskRepo and wrapping some logging around our TaskRepository so that we can see how that works. So still in our Plunk that we've been using this whole



time, let's look at our TaskRepository for just a minute. Now, our TaskRepository is just a service that has a couple of methods on it, get and save, that will handle database stuff for us. And if you're doing any size Angular application, you probably have several of these services that just kind of work. They do their thing and you don't really want to mess with them, but let's say, in this case, I want to add a level of logging to my taskRepo, or maybe some auditing or some additional functionality, but I know this works. And I don't want to start pulling apart this service; I want to wrap my additional functionality around this service that exists. And Angular actually has this built in, which is really kind of awesome. So all we have to do is use the decorator method that's attached to the module. And so let's create a new file and we'll call this repoDecorator. So the same thing we always do, let's do our iffy, and let's pull in our app. So app = Angular. module, and we're pulling in taskManager. Now I want to wrap taskRepo. And so what we're going to do is decorate taskRepo, so Angular. module actually has a method called decorator. And I can say I want to decorate TaskRepository, and if you look at this bottom here, we're putting out TaskRepository, so I want to decorate TaskRepository with something new. So basically, all TaskRepository was is a function, so I'm going to replace TaskRepository with a new function. And I just use the word replace instead of decorate, and so we need to be careful of how that works and make sure we're doing it right so we don't break a whole bunch of stuff. But, here's how this works. Angular is going to hand you the original service and it's going to call that \$delegate. And so, I now have the original service passed into my function, and if I just do a return \$delegate, I have effectively done nothing. I have just handed back to Angular what it handed me and TaskRepository at this point is unchanged. And it's especially unchanged because I'm not using this yet. So let's do that. Let's add this to my index. html and put it in there. Now one thing that is really, actually important here. Usually in Angular, the order of these scripts doesn't matter; in this case, it actually does. We need to have our repoDecorator underneath our taskRepo. Alright. So now all that's there, and what you'll see, not surprising, is that absolutely nothing has changed, which is exactly what you would have expected. So let's keep going. Alright. Now, what I want to do is I want to change the save method. I want to say \$delegate. save, and it's passing in a task, = function, passing it a task, and I want it to do some extra stuff. Now we're going to do some extra special logging for task with our task. name. Now once I've done that special logging, I actually want to call the original save method from our old taskRepo, but I just overwrote it, and that's how that works. So now I need to get back to the old save method and JavaScript closures are going to help us out here quite a bit, because I can do this: oldSave = \$delegate. save. So I can save the old version of this function; and then, once I've done my special logging, I can execute that old save. And even though this function's being executed out somewhere else, JavaScript closures say, hey, I still know what this oldSave is. So I still have access to this oldSave, and I'm going to pass this task in. Now when I refresh everything, assuming I didn't break stuff and I didn't, when I click task 1, it's going to complete it and it's going to save it. And when I save, it's going to say



Saving task 1 to the database, but it's also going to give me this console. log, 'Special logging for task '. So when I click task 1, now you see I've completed task, task 1 and I now have Special logging for Task task 1. And then, it went ahead and ran the oldSave. So, what Angular's letting me do here is take any service, any service, and wrap that in whatever way I want to wrap it in, so I'm not having to pull apart the TaskRepository to add a piece of functionality, or I could do notifications, or I could do auditing; I could do whatever I wanted to do here, and then, callback to the old method and know that I'm not breaking a whole bunch of stuff. Task 1 through task 4 are also working. If I click task 4, that's still working; special logging for task 4 works, along with all the other UrgentTask stuff. So that's how the app. decorator method works inside Angular to allow us to wrap services in Angular.

Facade Pattern

The next pattern we're going to talk about is the Façade Pattern. And this pattern is used to provide a simplified interface to a potentially complicated subsystem. We've all dealt with APIs or classes or things before, that just were nasty and very, very painful to deal with. The Façade Pattern gives us a way to stick something in front of that and make it very simple and easy for us to interact with that system. Now when you think about a façade, usually you think about the front of a building. And when you look at a building or a house when you're walking down the street, all you see is a very nice, clean front wall to a building. And there could be absolute, utter chaos going on inside that building or that house, but there's no way for you to know, because all you see is a very nice, clean exterior interface to the building. And that's what a façade does, is a façade hides all that chaos from us, or the complexity of the backend subsystem from us, so that all we have is a simplified interface to deal with when we're using an API or this application. Think about JQuery. JQuery is probably the most well-known implementation of a Façade Pattern. At its core, all JQuery does is sit on top of the DOM and give you a simple, clean interface to interacting with the DOM, instead of having to do all of the nasty JavaScript you might have to do to change the name of a button or the class of something. JQuery gives us a nice, clean, simple interface to deal with the DOM when we're trying to manipulate our web page.

Demo: Facade Node

Now let's take a look at the Façade Pattern in practice over on the Node side. So, real quick, as these demos and as the patterns start to get more complicated, I'm going to try and pull some of that complication out and keep just the essence of the pattern, so that we don't lose what we're trying to accomplish with the pattern in a whole bunch of implementations. So I'm going to, for this one anyway, keep



everything in the same file just so we can refer back to everything. If you want to see, or you're still curious about pulling things apart and having different objects and services in different files; go back and watch module 3, if you didn't watch module 3, and I do a lot of that over there. In this Façade Pattern, anyway, I'm going to keep it simple so that we don't lose the idea of the façade in all of the implementation of having everything split out. So that being said, I've taken our Task object that we've been using all along, and I've changed it around to be just properties. So this. name, this. priority, this. user, this. completed, all of those things are right here in Task. And because we don't always get to pick the things that we deal with, I've created this TaskService. And TaskService is not cool. This service has a series of functions that are in the Module Pattern, unfortunately not the Revealing Module Pattern, and it looks kind of gross, and it's hard to see what's going on, but that's what we deal with, right? That's the whole point of what we're trying to deal with with the Façade Pattern. So I've got these series of functions that allow me to interact with tasks, but I have to call them all individually, and there's no overlapping. So what it looks like now, when we're dealing with a new task, on line 27 I've newed up a Task, and then, when I want to complete a task, I actually have to call the TaskService and complete the Task, and then in fact it actually completed, I have to call three methods in order to get it all done. And that's just the way that API works, and the API kind of stinks, and that's just what we've got to deal with. So what I want to do now is take that gross chaos of dealing with this API and all these one-off function calls I have to make, and if you've been doing this for very long, you've probably come across an API that you had to deal with, or a service that you had to deal with, that made you do some things that you didn't want to have to do. And it was gross and you just didn't want to do it. Well, that's kind of what I'm getting at with this TaskService. So what I want to do is I'm going to create a wrapper, or a façade, on top of this service to make it look better. Now the difference between the Façade Pattern and the Decorator Pattern is I'm not adding functionality. The Decorator Pattern was actually adding and manipulating the functionality of the original task. In this case, I'm not adding anything. I'm just covering up and creating a better interface for exactly the same functionality. So that's the difference, really, between the Façade Pattern and the Decorator Pattern. So, let's do that real quick and clean this up a little bit. So what we're going to do is we're going to create a wrapper and we're just going to call it TaskServiceWrapper. And all TaskServiceWrapper is going to be is just a module. So if you remember from module 3, the Module Pattern just is a function that returns an object, and that object consists of things, and that's actually what our TaskService is is a module. And just a minute ago, I said that our TaskService isn't even using the Revealing Module Pattern that makes it kind of ugly to look at. Well, let's build our TaskServiceWrapper properly so that we're doing it right. And what we want to do is, if you look down here, we want to complete the Task and then, we want to notify that it's been completed and then save. So what we're really going to do in our TaskServiceWrapper is we're going to create a function that does completeAndNotify, and we're actually going to create a



function that's going to do that. And the Revealing Module Pattern says that I'm just going to return something that looks like that, so that I can see – actually we'll clean it up so it looks a little better – so I can see what methods are public and what's available out to the world. And now let's go create that function. So `var completeAndNotify = a function`, and then, we probably need to pass in `task`. And then I'm going to take all of this code that's in my `Main` and put it in my wrapper. So now, `var completeAndNotify` is going to call `TaskService` to `complete`, call `TaskService` to `complete`, and then, if it's completed, it's going to set the `CompleteDate`, notify of completion, and save. So, I can now get rid of all this code down here and switch this over to `TaskServiceWrapper.completeAndNotify`. Now one thing that I'm doing here, and this is demo, but instead of now creating a new instance of this `TaskServiceWrapper` and we're not creating instances, I know, but I'm just going to go ahead and execute this. So now `TaskServiceWrapper` is the return statement. So `TaskServiceWrapper` is now just an object that is `completeAndNotify`. Normally, if we're doing the full `common.js` pattern, it'd be in a different file and this would be `module.exports` and then I'd return the executed function, and that's essentially what I've done here. So now, my `TaskServiceWrapper.completeAndNotify` is going to execute and everything will just work. And if we watch our console. `log` when I run this, you'll see all these three things have happened. We set it a complete date and we've saved, and we'll see a notification happen notifying user of the completion of `Task`. All that stuff will happen, but now, I've got one line of code sitting in my `main.js`. So let's run that, `node task.js`, and there you go. Completing my task, `MyTask` was completed on, notifying Jon, and then you see `completed` is `true`, `completed date` is `now`. So that is the basic Façade pattern; it's just that easy. `Complete` and `notify`, we've just created a wrapper that wraps around our `TaskService`. And like I said before, we're not adding any functionality to `TaskService`. So that's the Decorator Pattern. Here, we're just maintaining the same functionality, but we're just doing it in a clean way, instead of having this chaos that we had before up in our `TaskService`.

Demo: Facade Angular

Now let's look at the Façade pattern over in Angular. Now I've taken everything we just had on the Node side and dropped it into a Plunk in Angular. So to look at it, we've got our `Task.js`, which now is just a series of properties; there's no methods or anything, nothing on the prototype like we used to have. It's just like this. I have a `taskRepo`, which is a `taskService`, which is this ugly, nasty looking service that we had over on the Node side. It's got a bunch of `complete`, `setComplete`, `notify` – all that stuff, that's on here. And then we have in our `script.js`, I'm creating a new `Task` with all of these things, and then when you check this checkbox, it's going to do the same thing we were doing before. So `complete task`, when you check the checkbox it calls `completeTask` and it passes



in the index. If you look at index.html – there we go, ng-click ctrl.complete(\$index), and it's going to do this same thing. It's going to complete it. If it's completed, it's going to set the complete date, notify completion, task.save, all of that stuff. And it's gross and we don't like it, but, you know what, it works when I click the check box – there you go; everything works. So let's build a façade over top of our taskRepo and fix that. So we're going to create a new file called taskServiceFacade. Now you wouldn't, in production, necessarily call this a façade. You'd either call it – you could call it a wrapper, or you can just call it whatever makes sense for your application. I'm explicitly calling it façade just because that's the pattern that we're talking about. Alright. So in our taskServiceFacade, do our iffy. Now I haven't talked about this up till now, but there is some discussion over whether we put that second here or in here. I like to put it inside like this, because when I'm teaching classes or I'm training people, I can say make a little face. So you've got an eyeball, a nose, an eyeball, and that way it's just an easy way to say hey, if something's not working, make sure you've got your face going on. So this is the way I do my iffies, it's not the way everybody does it, but that's either way will work. Now, inside our iffy, we are going to pretty much do the same thing we do over here on our service, right? We're going to create – well, first of all we got to pull in our app. So, var app = Angular.module('taskManager') and then we're going to create a function, taskServiceFacade = function. And then we're going to export – well, not export it, but we're going to do an app.service =, oop, not equals, because we know that won't be good – 'taskServiceFacade'. Alright. Now inside this function we're going to take exactly what we had over on the Node side, you know, this taskWrapper, or we called it a taskWrapper on the Node side, this taskServiceFacade, is just going to be one function, completeAndNotify that's going to do some stuff. Now we've got to pull in our taskRepo. Now taskRepo is 'taskService', all lowercase like that, so we're going to tell Angular, hey, I need taskService, and then, since it's differently cased we've got to paste some stuff in there. Okay, now, that's the wrapper. It's the same as it was over on the Node side. Basically, we've just built a method called completeAndNotify that tasks a task and then does the stuff that we want done on the taskService. And then we return, using the Revealing Module Pattern, that function. Now in script.js we're going to copy this, because in script.js we need this passed in. So I'm just going to add it for right now. Ideally, we would be replacing it with taskService, but you're not always going to replace all of the functionality of taskService; in an ideal world you would. But you know, can't always do that. So, in this case, I'm just going to add my taskServiceFacade. And then instead of all this code, we're just going to delete all that and do our completeAndNotify on this task that we pulled up. And it's taskServiceWrapper.completeAndNotify on myTask. So what we should see is this same thing happen again, but now with just this one line of code. So before I get carried away though, we do need to add this JavaScript file, copy that into our index.html. That's going to be important. Alright. Now, let's refresh, hit MyTask, and there we go. Completing task: MyTask, MyTask completed; everything's done, but I got all of that junk out of this function



by wrapping my taskService with this façade. So that's the Façade Pattern. There's no magic here. It might seem even a little simplistic, yes; we're just taking code out of one place, and we're putting in someplace else. And we might add a little bit more to it, but remember, the goal here isn't to add functionality; the goal is to mask chaos. And that's what we're doing here is we're taking all those function calls we had to do in order to interact with that one service, and we're hiding them inside a façade so that it simplifies our controller. And really, if we're talking best practices, you want to keep your controllers and your business logic separated out, and keep your controller code as small as you can just so that it's easier to read and much more maintainable. So the more you can pull things out and hide grossness with some facades, the more maintainable your code is going to be.

Flyweight

The last structural pattern we're going to talk about is the Flyweight pattern, and the Flyweight Pattern is used to conserve memory by taking portions of an object and sharing that across objects. So think about it this way. Our tasks have a lot of non-unique data. If you look at all the tasks that we might put together in our task management program, there's only a small number of users, there's probably a small number of projects, we had priority; there's only a few options for a priority, and yet we replicate that data. So if I have 5, 000 tasks, which over the course of a year is probably easy to get to; if I had 5, 000 tasks, that string, Jon, is going to be shared and replicated 5, 000 times, because all of the tasks that I work on, I usually, it's me working on them. So I've replicated that one string 5, 000 times. And so what flyweights do for us is they share data across objects. So instead of replicating that name, Jon, 5, 000 times, my Flyweight Pattern allows me to share that across all the objects so I only do it once. Kind of like when we were doing the object prototype, instead of creating the functions a whole bunch of times for every task, we add them to the prototype. Well, Flyweight is going to let me do kind of the same thing, but in a slightly different way. The overall result is a smaller memory footprint. Now on the browser, sometimes memory's not as important as we would like it to be, and especially once it comes down to the mobile device. You want to keep your memory footprint as small as possible, and in some cases, the Flyweight Pattern's going to help us reduce that memory footprint. And so, think about it; it's like a boxer, a flyweight. The smallest possible thing that you can get, that's what we want to get. But this is only useful, so this is the big caveat to the Flyweight Pattern – it's only useful if you have large numbers of objects. If you've got 100, 500 objects, implementing the Flyweight pattern, unless your objects are really large, is not going to do you a lot of good, because there's some overhead to implementing the Flyweight Pattern. So when I say a lot of objects, I mean a lot of objects. And we'll look at what that looks like when we do our demos.



Demo: Flyweight

Alright. Let's look at the Flyweight demo, and this will start to make a little bit more sense. Now because of the complexity of the Flyweight Pattern, the setup for this demo's going to be a little bit more complicated. So let's take a minute and walk through what we've got going on, so that when I start making changes everything will make more sense. So to start with, right here, we have our Task object. Now our Task object looks a little bit differently now because I'm adding some more things to it. We've got a name; we've got priority, project, user, and whether or not it's completed. There's more pieces of information on our Task object. Then, I've also gone a little bit deeper and actually made a TaskCollection. So what this collection is going to do for us is, I've got three methods and you know that because I'm using the Revealing Module, which says I've got an add, a get, and a getCount. And what those three methods do: the add method adds a task to the collection; it actually will new up a task based upon some data, and then add it to my tasks collection, which is really just an object, and the key is my name, the name of my task, and then get is going to go pull it back out, and then my getCount I'm using just so that we can do some metrics later. So now, what we're doing is I've created an array of the different types of projects. So a task can have no projects, it can have courses, it could be training, or it can be a project. So these are the different types of things that I work on on a given day. I'm either working on a course, I'm doing some training, or I've got some project work going on, priorities 1-5, and I've got a set of users of people I might be assigning tasks to, so these are others. So it's me, Erica, Amada, Nathan. These are people I might assign tasks to, so the user on a task, and then, completed is either true or false. And I've created, in this for loop, just a loop that's going to create random tasks. So I am going to create 10,000 random tasks so that we can see what the impact is of what we're doing. I'm able to, using `process.memoryUsage`, to see how much memory we're actually using in our application. So I can do `process.memoryUsage` and get the current memory that's being used by our application. I'm pulling in the initial memory usage, then I'm going to create 10,000 objects, and then once those 10,000 objects are created, I'm going to pull in my `afterMemory` so we can see how much memory I've used over the course of creating 10,000 objects. And then I'm going to write out to the console how much memory we've used, it's in bytes, so let's do megabytes, and then `console.log("tasks: and just how many tasks we've created. So if we run this just by doing node main.js, what you see is we've used 3 MB of data and we've created 10,000 tasks. If I change this to 100,000 tasks and run it again, now we've used 28 MB, which I've multiplied it by 10, that kind of makes sense, and created 100,000 tasks. Now, let's implement the Flyweight Pattern on this so that we can see what impact that would have on our memory usage as we go. Now remember, I said this is only used for things that are fairly complicated, and what you might see is that when we add the overhead of the Flyweight, this initial memory might go up a little bit. And the reason is I'm having`



very simple tasks. It's basically just a set of strings. When things get more complicated, that's when the Flyweight pattern starts to make a little bit more sense. But let's run through creating a flyweight so you can see, kind of, how this thing works. So let's minimize our TaskCollection here, because we're not really worried about that any more, and let's create a flyweight factory, but really you can call this something else like subtask, or you know, partial task or whatever you want to call it. But I'm going to be explicit and I'm actually going to call it a FlyweightFactory and this is going to be a function. And realistically, we're just going to immediately execute it so put our face in there, because we want the FlyweightFactory to actually be the factory, itself. And the FlyweightFactory is going to do two things. It's going to create a new flyweight and it's going to return a flyweight, and then, we also want to put a count on it just like we had before. So let's start by setting up our collection object just the same way we did in our task collection; see, we've got our var tasks there, so we'll do our var flyweights, and then we're going to ask for a flyweight. Now a flyweight is a subset of a task for us, so if you look at the Task object, the only thing that's unique, the 100% unique for a task, is this. name. Everything else is shared across one or more tasks, and we know that because down here I'm creating random values for the rest of these things. So, the only thing that's unique is the name, so everything else can become part of a flyweight. And so what we're going to do is we're going to create a get function that's going to be responsible for getting a flyweight for us. And this get function is going to take all of the things that are not unique to the task. So in our case, it's the project, the priority, the user, and completed. So I'm going to pass those things in to my FlyweightFactory and then my FlyweightFactory is going to check and see if that exists in its collection of flyweights. And if it does exist, it's going to return it. And if it doesn't exist, it's going to create a new one. So right here, if not flyweight – if this collection of project, priority, user, and completed does not exist already in our flyweight collection, then, we're going to create a new Flyweight using those four things. And basically, all that Flyweight is – now we haven't created that yet – all that Flyweight is, is those four things. So let's create that real quick. So now we've got a Flyweight object that takes project, priority, user, and completed, and just does this. on them. And basically, we took these this. 's and we moved them down. So I'm actually going to comment these out; there we go. Now, if that flyweight already exists, it's going to return the flyweight that exists already. So now what we need up here in our Task is to add the flyweight. So we've removed priority, project, user, and completed, and we've added a flyweight, and basically, we're just saying FlyWeightFactory, get me a flyweight with this project, priority, user, and completed. If that flyweight exists, hand it to me; if it doesn't exist, create a new one, and that's what we're going to do. So the last thing we want to do on FlyweightFactory is implement a getCount, because we want to know how many flyweights there are. So basically, all we're doing is var getCount is now a function that's going to count the flyweights. And then we're going to return an object in our Revealing Module Pattern, our get and our getCount. So up here we're using get FlyweightFactory. get and then, down here



we're going to add a line for getCount. So right now, I'm logging the number of tasks, now let's log the number of flyweights. Real quick, copy/paste, FlyWeightFactory, and then capitalize the W; now let's do it. Alright. Now, I've got 100,000 tasks, but only 160 flyweights, which means that data is shared, the priority, project, user, completed, any combination of those things, was only shared 160 times. So I've cut down the amount of memory that was used. Notice, I was 28, now 25. So not a huge amount, but I've cut it down. If this was maybe a little more complicated, and we had more, it might be more, but right now, implementing the Flyweight Pattern on just this simple task has saved me 3 MB of memory, which is pretty decent overall. If I add another 0 to this, notice I'm up to 279 MB of data used. And if I drop my flyweight and I go back to doing it the old way, I go from 279 MB used to 319 MB used, which, on a percentage basis, is actually fairly significant. So the idea of sharing pieces of a task or an object, in this case a task, across tasks to save myself extra memory can make, at least somewhat of a difference, and it depends on the complication of your object. Now one thing we didn't do, and we could go through the motions of it, is right now your interface has changed. Right now we've got a flyweight something, and what you would do is you would add on the prototype for Task, prototype, let's say, getPriority, and you would return this flyweight priority so that you don't have to expose the fact that you're using a flyweight. You do everything with getters and setters, and the fact that you're using flyweights is not visible to you. Alright. So that's the Flyweight Pattern and I admit, the Flyweight Pattern of the three we've looked at on the structural patterns, is probably the least likely to be used, but when you need it, you need it. So if you're doing things with large datasets, or you have many instances of an object, the Flyweight Pattern might be a good way for you to cut down on your memory overhead to help improve your performance.

Summary

So in this module we were dealing with Structural Design Patterns, or patterns that are intended to change or at least alter the view of an object. Creational Design Patterns we talked about first, that we're dealing with creating with new instances of objects. In this case, the Structural Design Patterns, we're dealing with altering the composition of objects. Now we talked about the Decorator Pattern, which allowed us to add additional functionality onto an object without having to change that object itself. We looked at a couple different ways to do that with either additional logging or adding new functionality to take a Task to an Urgent Task. We looked at the Façade Pattern, and we looked at how to hide a lot of junk and a lot of, maybe, nasty API implementation behind a clean wrapper, or a façade, to give us a nice view of an API. And we ended with the Flyweight Pattern, which we talked about how to use that to create pieces of objects that can be shared across our object collection to help reduce our memory footprint, if needed, when we have large numbers of objects. So that's it



for the first two types of design patterns. We've got one type left and these are behavioral, which are going to be more about interactions between objects and how to communicate across objects.

Behavioral Design Patterns

Introduction

In this module, we're going to start talking about behavioral design patterns. Now, this is the last of the three types of design patterns that we're going to talk about in this course. Behavioral design patterns are concerned with the assignment of responsibilities between objects and how objects communicate. So while creational design patterns are very much interested in newing up an object such as the constructor method or the factory that were handing me back new instances of an object, and structural were more about adding functionality to an object, and they were all self-contained within an object or wrapping an object and things specifically. Behavioral design patterns break out of that single object mentality and start concerning themselves with interactions between objects. Now like we said, behavioral patterns deal with the responsibilities of an object, and they'll do things like help objects cooperate. If we've got multiple objects that are trying to accomplish tasks, we have patterns that are going to help objects cooperate with each other to all achieve the same goal. We can also assign clear hierarchy for objects. We can set one object as the object of record and everybody else watches that object to see when something changes and then they'll go and do the things that they need to do. We can also encapsulate requests between objects to make sure that requests are being made appropriately, and we can either alter those requests as necessary or we can change requests to suit our needs. So let's start looking at what these patterns look like and the types of problems that we can solve with them.

Observer Pattern

The first pattern we're going to talk about in the behavioral patterns group is the observer pattern. Now, this pattern allows for a collection of objects to watch one other object and be notified of changes that happen. The benefit of this is it allows for a loosely coupled system. Now, everything's not tightly coupled together and you're not actively calling out to different systems, you can allow one object to say, "Hey, I'd like to watch this object, " and that object will then notify it of any changes. One object is the focal point, so in some sense, it's a little coupled because you have one object that knows everything that's going on, and then you have a group of objects that's watching for changes to that



object. Now, this one's a little bit more complicated and we've got a lot of moving parts, so let's take a minute and look at this in slides with boxes so that we can see what's happening and all the moving pieces. And keep an eye out for everything we've talked about up till now because we're going to use some of the patterns we've talked about up till now to implement the observer pattern. Now, in the example we're going to use, we're dealing with tasks so we've got our Task objects, and what we want to implement is three services. We've got a logging service, a notification service, and an auditing service. Now, if you remember, over the course of this course, we've dealt with this notification service before where we wanted the Task to notify people or we wrapped a logging service at one point, and they were always either decorated on or tightly coupled into the Task itself. Well, in this case, I want to decouple this so that the Task service lets these other services know what's going on, but not in a tightly coupled way. The vocabulary we're going to use for this is three words. The first one is observers. The logging service, notification service, and auditing service, in this case, are called observers. They are going to observe Task and watch for changes. Subjects is what the Task's going to be. And actually, we're not going to mess with Task specifically. In our example, we are going to decorate Task with a Subject, and the Subject basically comes down to a list of observers, and a Notify method. So what we're going to do is decorate Task with a list of observers and a Notify method. Now, what's going to happen when something changes on Task, Notify is going to send a message to each of these three services, and when I say send a message, basically what it's going to do is execute a function that these three services have supplied to Notify. So logging's to supply a callback effectively to the ObserverList and then when the notify is called, it's going to execute the callback that logging gave to us. So these are the pieces and the moving parts involved with what we're working on. So let's take a look at a practical application of this so that we can see all the moving parts and how this works.

Demo: Setting up the Environment

All right, because of the complexities of what we're doing here with the observer pattern, I'm going to break out this demo into four or five different clips, and we're going to just take a piece at a time and set up each individual piece and then we'll see it all working together. So let's start by just setting up our environment. Now, all we're starting with is our Task, so I have a Task, and it's got name, priority, project, user, all that stuff on it, and we've got two methods on the prototype, complete and save. So what we're going to do is we're going to create a new file called main.js, kind of the same way we've been doing this all along, and we're going to require ./task. And we're going to set that equal to Task. Okay, so we're pulling our Task in, and we are going to create and save a Task. So var task1 equals new Task. And really, the only pieces of information I'm interested in right now are the name and the user because ultimately what we're



going to do is notify the user when something changes, so we're just passing in name and user. And then we're going to do a task1. save. So now when I run this, we're just saving Task: create a demo for the constructors, and that's coming out of save. All right, so we've got our Task set up. The idea here is we want to add three observers to this Task so that when I run save, now those observers, they're notified, and they get to do their own thing.

Demo: Creating Our Observers

All right, now let's create our observers that want to observe Task. And if you recall from our slides earlier, we've got three observers. We've got the logging service, the notification service, and the auditing service, and all three of these services want to be notified when Task changes. So the way we're going to do that is we'll keep it simple just for the purposes of what we're doing here, but complicated enough so that you know it's working. We're just going to create a service, and that's going to be a function. Now, on our notification service, we're going to have two things. We're going to have a message, and in this case, it's the notification service, so we're notifying. And then we're going to have a method. And this method is going to be update, and update is going to be what's called when Task changes. So when Task sends out a notification to all of its observers, update is what's going to be called. So in this case, we're going to send in Task, and we're just going to write out a simple message to the screen saying message, so we can look at our closures and see that, hey, var message is coming through. task. user, for task, task. name. So all of our notification service for right now is going to do is just write out to the console, hey, Task changed and I'm letting somebody know about it. Now, we're going to do exactly the same thing two more times with the logging service and an auditing service. The only difference between these services for the purposes of this demo is that the message is different. If you just copy and paste notification service and change message to logging and auditing, they're pretty much the same thing. I just want you to see three different observers, and then we can add and remove observers later so that you'll see how that works. Okay, now we've got some observers, and those guys are ready to go. We need to new some up because we're creating these as objects, so we'll actually new these up down here at the bottom. So, I've got notification service, my logging Service, and my audit service all ready to go, all set. Now, I just have to be able to let them know that something's changed so let's go look at creating our Subject with their ObserverList.

Demo: Creating Our Observer List

All right, the next piece of this observer pattern is the actual ObserverList. And if you recall from our slides earlier, the Subject, and the Task is going to be our Subject, has this ObserverList. And that's going to be the list of observers that



Task is going to notify when it's done. And this is its own separate thing with add, and remove, and indexOf, and all of those methods on it. So let's take a minute and build out our ObserverList before we can add that list to our Task object. So back in our main.js file, I've got my Task, I've got my three services. The next thing we're going to add is our ObserverList. So our ObserverList is just going to be an object. It says list, but really it's an object because we're going to add a whole bunch of functions to it. So we're going to set it up with the constructor pattern and create it as a function. And really, at its core, the ObserverList is going to be just an array and that's it. Now we've got our list. Now that we've got our list, we have to add all the methods to it. So the obvious methods for our ObserverList are going to start with add. I want to add an observer to the list. So, we'll do a prototype. ObserverList, the method is going to be add. And we are going to pass into that an object, and in this case, we're just going to return and push whatever is passed in onto our ObserverList. So as observers are registering themselves, whatever they send to that observer is just going to be pushed onto that list. The next thing we're going to add is get because if we're going to add, we also have to be able to get. And so, here we've got our get method and we're passing in the index, and then we're just handing it back. So, for right now, for the ObserverList, that should do it for us. We're going to add maybe one or two other methods later, but I want to get to a working point fairly quickly here, so for right now, for our ObserverList, we've got an add and we've got a get, and that's really all we need. So let's take that and leave it there and now let's turn Task into a Subject.

Demo: Adding Our Subject

All right, now the last piece of the puzzle is our Subject. We've done our observers, we've done our ObserverList, now let's create our Subject, so we can make this whole thing work. Now, what we want to do is to make our Task a Subject, but if you've been watching this course long enough, you kind of know my feelings on messing with things that already work, so I don't want to make Task a Subject by adding ObserverList and Notify to Task because Task already works, I don't want to mess with it. So what we're going to do is we're going to wrap Task with a Subject and make it an observable Task. So, let's take a minute and let's build out an observable Task with an ObserverList and this Notify function. Now, the way we're going to do that, if you remember from our conversation on the decorator pattern, is we're going to create a new thing called an observable Task. So, let's minimize this stuff, fold it up, and get it out of our way. Now, we're going to do an observable Task, so the var ObservableTask equals the function. Working the same way that Task does, so it takes data and then call out to Task. So Task.call, and remember what call does is it allows me to pass this into it and then whatever parameters. So I am just passing the this context of ObservableTask off to Task, so now ObservableTask has, everything Task adds to this gets added to ObservableTask. And then I also want to do



something with the observers, so I'm going to add to my ObservableTask, this Observer's list. So this. observers equals new ObserverList. And basically I'm just newing up an empty array. If you look at ObserverList, it's just an empty array, this. observerList, that's all it is. So, now that I've got an observable Task, I'm going to add a couple of methods to its prototype so that we can start dealing with observers, and the first one is going to be to add an observer. So in this case, we've added an addObserver function and we're going to pass in to that function observer and then just add whatever's passed in to our ObserverList, and this is what's going to be executed when we do notify. So, notification service has got to pass in, if we go back up to notification service, what it's going to pass in is this. update. It's going to pass in update, and then that's what's going to be executed when we do notify. All right, so I've got ObservableTask, and I've got addObserver. Now, we want to added notify. And notify's going to be a little bit more complicated. So we've got a function called notify and we're passing in a context, so whatever the context is that we want to do. In our case, we're going to pass in task because Task calls notify, it's just going to pass itself in as the context so that when my alert gets called, update, notice I'm asking for a task, I'm going to do task. user, so my context in this case is always going to be task. Now, I'm going to loop over that array. The first thing I'm going to need when I loop over the array is how long my array is. And we didn't implement a count yet, so let's implement a count. So ObserverList. count, just to get the length of how long our ObserverList is, so we've added the prototype of count, which is just going to return observerList. length. And I don't have to hide all this stuff. I could just access that array directly, but I'd rather go through the motions of adding helper methods so that we're protecting our array a little bit. All right, so now I've got a count. Now, back down here I've got my observerCount, and now I'm just going to loop. So just going to do a simple little for loop that's going to iterate over my observers. And then, for every observer, I'm going to go and get whatever they've passed in, and then I'm going to execute it with my context. So basically what that means is I have allowed objects to add themselves to me, so addObserver, I'm adding observers, and then when I hit notify, I'm just looping over that array of observers and calling whatever's been passed to me. I'm not checking to make sure it's a function. I probably should do that, but add that in if you would like to. All right, now I have to call notify, and I have to figure out what I want to notify on. So if we go back and we look at Task, it's got two methods. It's got complete and save. So, let's say when save is executed, so whenever you save something, I want my, scrolling back to the top, I want my notification service, my logging service, and my auditing service all to know about it. So, I am going to overwrite my save method on Task. So ObservableTask, my method name's going to be save, and that doesn't take any arguments. So if you remember how to do an overwritten method, the last thing we're going to do is actually call Task. prototype. save and we're going to do a. call so we can pass in our this context. And since we don't have any arguments, we're not passing in any arguments. Now before I save, or maybe after I save, you could do it either place, I want to notify. So I'm going to do a this. notify. And I am, for my context,



remember, I'm going to send my task. So I'm just going to send in this, so that that way, whatever method is getting called is getting called with my this context, not as their own this context, but just so it has access to my this. And in this case, what we're doing up here is Task. So, now that I'm doing a this. notify, everything that adds itself to my ObservableTask is going to be notified. The last piece of this is to change task1 into an ObservableTask. There you go, now nothing really should change. Everything should still work with my task. So now task1's an observable Task. It's got notify on it, all that stuff's done. Now we just have to register our services on our task, and the way we're going to do that is we're going to do task1. addObserver and pass in whatever we want observed. So, in this case it's not. update. Not an executed not. update, just pass in the function itself. We'll copy that and add two more because we want to do our logging service and our audit service. So now when I save, I should get a notification, a logging, and an audit all working for me. So if we look at this and we run it, if I do node. main. js, what we'll see is our three notifications, we'll see notifying user, for task, task, we'll see logging user, for task, task, and we'll see auditing, user, task, task. So let's run this and see if that works. So there we go. Notifying, logging, auditing, and then the Task. So this is the observer pattern, this is how that works. task1 keeps a record of all the observers and then notifies everybody that they're there. There's one more piece of this I want to do, though. Let's add an option to remove observers so that an observer can opt out of receiving notifications.

Demo: Removing Observers

Now, the last piece of the observer pattern is the ability to remove observers or allow an observer to no longer get notified of changes that are being made. There's two things we need to add to our observerList in order to make this happen. One is the ability to remove at a spot, and we're going to add a method removeAt that takes the index, say hey, I want to remove at this and then splice our array by removing one item at that index. Now, the other thing we need to know is where that index is. We have to be able to find an observer in our list. So the way we're going to do that is with an indexOf function. And we're going to basically just loop over our observerList until we find something that is equal. And we use triple equal, so it's the same thing. I'm looking for something that is the same, instance-wise and everything, of this object. And so, when I find this object on my observerList, I'm going to return that, and then I'm going to splice it out. So let's, on our ObservableTask, add a removeObserver. So, ObservableTask, and our method name is going to be removeObserver, and we're going to take in what that observer is. And the functionality here is really quite simple. We're going to take those two methods that we just created, we're going to drop them in here. So we're going to removeAt wherever that observer is. So, we're going to look for the index of that observer, and then we're going to remove that observer from the list. Now, let's try that out and make sure it works. Down here at the bottom, I've



got my three addObservers, and then I'm going to do my save. I'm going to do a task1.removeObserver, and let's remove our auditing service because that sounds like a bad idea, and you never want to remove auditing, but we're going to do it anyway, and we're going to run save again. So now, the first time I run this, I should see all three get updated. After I've removed an observer, I should only see two get updated. Real quick though, we need to change this to audit.update so that we're removing the right thing. Here we're adding an addObserver, audit.update, so we remove audit.update. Now, let's try this out, and what you'll see when we run this is that first I have notify logging and auditing, and then I save the task, and then I have notifying and logging with no auditing. Now, if we go back over and we change this to our ls.update, and we run it again, this time we've got notifying, logging, and auditing, and then notifying and auditing with no logging, so that's how that add and remove works. That's the observer pattern. That's how this whole thing works. I've got three things that want to be notified when the change is made, and essentially I'm just adding a function to a list of observers that our task is executing every time a change is made.

Mediator Pattern

The next pattern we're going to talk about is the mediator pattern, and the mediator pattern is an object that controls communication between objects, so really neither object has to be coupled to the other objects. If you remember the observer pattern, the observer didn't really know much about what was going on with the Subject, but the Subject knew about everything. So the mediator pattern is a little bit of a resolution to that thought. Think about it like this, the mediator pattern allows for a loosely coupled system. Neither side has to know very much about the other side in order for this to work. And one object manages all of the communication. Think about it just like a mediator in a conflict resolution. If you got a strike going on, and they call in the mediators, well, the mediators are the one that manage all the communication. So everything that needs to be communicated goes through the mediator and then they send it out. This really allows for a many-to-many relationship because I can have several objects publishing events and several objects receiving the notifications from those events, and it all flows through one spot. So if we look back at the observer pattern, the Subject was sitting on Task and the ObserverList was there and Notify was there, and it was responsible for pushing everything out to the other services, and that can be maybe a little hefty. If you watch the flyway pattern piece, we talked about memory management, and adding more stuff to something I'm going to replicate a lot of maybe isn't the best idea. Well, the mediator pattern splits all this out. So now the Task pushes events to a mediator, and that mediator looks at everybody who's subscribed to that event, and sends notifications out to everybody that subscribes. And the mediator can do a lot more than just that one shuffling of information. They can manage that communication and make decisions based upon information that's



coming in.

Demo: Mediator Pattern

All right, so let's build out this mediator pattern using all the same pieces we did from the observer pattern. So if you look at what we've got, we've got our Task, which is the same Task we've kind of been using all along. Just name, user, completed, all that with the two functions, complete and save. And we're going to leave that intact and we're just going to use that the way we have been for the last couple of examples. Now we've got our three services, our notification service, our logging service, and our auditing service. And those are the same exact thing we used before, so they just each have an update method that's just going to write out to the console. Although, in practice, you would call a service, or you'd call an API, or go to the database, or something. That update method would do something along the lines of whatever the service is for. Now, we're creating a new Task. Create a demo for mediators with user of me, an instance of our three services. Now, the piece to build out is our mediator. and remember, our mediator is just going to kind of sit in the middle, and it's going to manage everything for us. So the first piece of building out the mediator is to get us set up, and we are going to use the revealing module pattern for our mediator. So, we want mediator is going to equal and we're going to just wrap it an IIFE. So function, and then we're going to immediately execute it, there we go. Now, our mediator is going to have a couple of pieces to it. We're going to have channels, so you can publish or you can notify to a specific channel. And that's really what everything's built around here, so we've got channels, and that's going to be an object. And then you have publish and subscribe, so let's do publish first. Or actually, let's do subscribe first. And that's going to be a function, and you are subscribing to a specific channel, and just like over on the observer side, we're going to have a context, and then we're going to have a function. So this would be like the callback. I don't like to call it a callback in this situation because it's a little different. Although, some would say it's exactly the same thing, but I'm going to call it a function that we're going to execute. Now, when we subscribe, we're going to either add the channel or we're going to just add this subscriber to the channel. So the first thing we want to do is actually check to see, hey, if this channel doesn't exist, well, let's create that channel. Now, once we've got that done, now let's actually add to that channel. So we'll do mediator.channels Now, you may be tempted to think that this is an array because I'm using bracket notation, but this is an object, so don't get confused and say hey, channel's an int here, because it's not. Channel's a string here, and I'm just using bracket notation on the objects, not bracket notation to pull in an integer. Since we're really having a list, it's easy to get confused into what that would be. Now, then we're going to push. Now, the subject of that specific item in the object is going to be an array, and we're going to push onto that array, the context, and the function. And that really is all that is happening



on my subscribe. So recap, subscribe is just a function that's going to create a channel if it doesn't exist already and then push the context and the callback function, or just a function, onto an array by that channel. The other piece of the mediator is the publish. So it's pub subscribe, right? So, publish equals a function, and this time we're just going to take the channel. Hey, channel, go. Now, we are passing more into this than just the channel, but I don't know what we're passing in because we're calling a function, it could be anything, so I'm not going to restrict it right here. And I'll show you how we're going to deal with that here in just a second. So the first thing we want to do is, you know, make sure the channel exists. So we'll do a little bit of checking. We're not going to do a lot, but just so you can see how it works. You can put that in there. Now, if that channel exists, we want to loop through everything that is involved in that channel, so we're going to do an if. `i equals zero, i`. So we're going to go less than mediator.channels.length, so however long that array is, `i++`, not if, for. Okay, now for every subscriber to that channel, we're going to execute that method that was called. So for every subscription, mediator.channels for every subscription, we are going to execute that method. So, `sub.func`, the function that was passed in, `apply`. And the reason we're doing `apply` here is we're going to pass in the context, `sub.context`, so whatever it sent to us, and then the args. Now, the args we haven't gotten yet, but this is, the args are going to be everything else that was passed in, so when I publish to a channel, I'm also going to send a whole bunch of other arguments with it. We need to get those, so outside of the for loop, those aren't going to change. We are going to add `var args` and it's going to be a slice of arguments, and the reason we're slicing off arguments, so arguments is all of the arguments that are passed into the function. We're going to hack off the first one because we already know it, that's channel. Everything else we want to put into our arguments, and we're going to pass that into our `apply`. So basically what we've just done is we are calling the function that was passed into our mediator with whatever context it passed in and whatever arguments were set. Now, that's our whole mediator. Remember on our observer side we had a whole bunch of stuff. Mediator's much simpler because it's really just going to pass things around. You can make the mediator a lot more complicated, but we're going to keep it pretty simple for right now just so you can see how it's done. Now, I've got my methods, our revealing module pattern says I've got to now return those methods. I'm returning the channels, subscribe, and publish. And I'm returning the channel so that I can get access to it again later because I'm just declaring it as a variable and I'm doing `mediator.dot` everywhere. So just a slightly different look on the revealing module pattern that we've had up till now. Okay, so now that we've got our mediator, let's fold that up so we don't get confused. I've got my task, I've got my services. So now let's start adding things into my mediator. And to add that, we just do `mediator.subscribe`. And I'm going to subscribe to the complete channel. I could subscribe to whatever channel I want. It doesn't matter, remember? Because if I pass in a channel that doesn't exist, it's just going to create it. So when I do a `mediator.subscribe` complete, it's creating that channel, and I'm going to pass in my notification service, and I want



to do update. So when it executes the update function, remember, it's doing apply using that notification service. And let's do two more. Let's do ls and audit. All right, so all three of those are subscribed and ready to go. The only thing that I still have to do is publish to the complete channel. And we're going to decorate task1. We're not going to do full subclassing, we're just going to decorate task1 so that it's calling to this channel whenever complete is happening. So, let's do this. task1. complete is going to now equal something else and I'm overriding that function with a new function, and the first thing we're going to do is a mediator. publish. And we are publishing to the complete channel, and we're passing in this, so the context of task1. And so remember, if you look up at our actual publish service, so let's blow this back out, if you look at our publisher, it only takes channel. We didn't pass in the task. Well, we're passing it in, that's why we pull out the rest of the arguments, and then pass them in when we do the function. apply because now I'm passing in that task that's going to be necessary for my notification service, logging service, they're expecting a task to be passed in. Okay, so mediator. publish, this, and that's great, but now I have to, I've overwritten my task1. complete, so we've got to call task1. complete. That's actually coming off a Task. prototype. complete, and we're going to do. call and pass in this. So I'm calling the complete function with the this context. And now we're doing this to complete instead of save, so we'll change this to complete. Okay, so recap real quick. We've got three services: our notification service, our logging service, our auditing service. That's the same that we had in the last one. We've built this mediator and all the mediator is for this. You can make these a lot more complicated, but all it is for this is just a subscribe method and a publish method, and then a list of channels, and that list of channels just has a channel name with an array of subscribers. And so, mediator. subscribe adds our services to the subscription, mediator. publish sends a message and a context to everybody that subscribed to that channel, and then task1. complete. So when I run this mediator, you'll see notifying, logging, and auditing, and completing task. So I have subscribed to a complete, and I have now executed those three subscriptions when I did my task1. complete. So that's the mediator pattern. That's a simple version of the mediator pattern. You can make this significantly more difficult if you want to and put all sorts of decision points in on everything, and you can manage all that here in your mediator, but beyond this basic framework, pretty much the sky's the limit. But once you get this framework to work, you can do whatever you want with the rest of this pattern.

Command Pattern

Now, the last pattern we're going to talk about in this course is the command pattern. And the command pattern is just a way to encapsulate the calling of a method as an object, as its own individual thing. And the reason why you would do this is it allows for fully decoupling the execution from the implementation. Up



until now, when we've had services, I had to do service. save, and I have to understand in the Task object that service has a save method, and here's the parameters necessary for that save method. And that's a little coupled. We're going to decouple that and make it so that we're just executing an implementation. And this allows for a much less fragile implementation. If I want to change something on my service side, I can, and not have to worry about if I change a word or change an execution of a method, I can deal with that over on the service side. And also, if you read the Gang of Four book, it allows undue operations. I can store a state before I execute a method, and then revert state if possible, and I can handle all of that as part of my execution. And the last piece is it supports auditing and logging of operations, so I can log what operations are done in one central place, I can log what operations have done so I can rerun them if necessary or audit them. And if you've been paying attention to my icons over on the side for each one, I picked the USB symbol for this one because that's kind of the idea that we're looking for. I want a standard interface to execute everything, and the command pattern kind of lets me do that. It lets me pull out the concrete implementation and just give me a generic interface, kind of like what a USB is, a generic interface into my services to execute commands as needed.

Demo: Command Pattern

All right, now let's take a look at how to implement this command pattern on one of our repos. So I have pulled over from module three, the task repo we were using when we were dealing with our task object. And basically, all this repo has is two methods, the get method and the save method. And those'll do things for me as we go, but we want to do is we want to eliminate the need to know it's got a get method and a save method, and work that a little bit differently. So what we're going to do is we're going to add to this repo an execute method, and this execute method is going to take the name of a method that we want to execute. And I don't care what it is, it could be anything. And then it's going to run some stuff. So what our repo. execute is going to do is it's going to actually call the method for us. So if repo then we're going to call that with the. apply. Now, if you've been watching the whole thing, you'll see I'm using call sometimes, and apply sometimes, and there's a reason for that. I like to use. call and I can pass I the context, but. apply allows me to just pass in an array of parameters.. call, you have to list out each parameter individually.. apply lets me pass in the array of arguments right here in the second parameter of. apply. So that's why sometimes I'm using. call and sometimes I'm using. apply. It depends on the context of what we're doing. Speaking of which, I need an array, and so that's what we're going to put right here is the same thing, I think we've done this once before. var args equals the slice off of array. And I've spelled it out, Array. prototype, just so it's obvious. You can just do that, array. slice. call, arguments. I'm just being more explicit. And I'm cutting off the first one because we already are using that one



somewhere else. Now, if that exists, I'm going to call it, and if it doesn't, I'm not going to call it. And then we're just good. Otherwise, I'm going to return false. So the way I deal with this is I call `repo.execute` in order to execute a method on that repo. And in this case, let's do `save`. And I can pass in a Task object that I want to save to the database. Let's just pass in a task with an ID, a name, and whether or not it's completed. And that for right now will be the task that we're going to work with. Now when I run this, you'll see we get Saving Task 1 to the database. So the way the command pattern works is I just pass in as a parameter the method I want to call, and the object that I want to call it on. Now, this seems to be a little bit more complicated, so let's build this out a little bit, so you start to see why we might use the command pattern. We'll start by adding an actual repo of task. So we'll just do this. We'll say `tasks`, and that's going to equal an empty object. And we'll do commands, and that's going to be an array of all of the commands that are executed on this repo. Now, when I execute `save`, let's actually save to my repo what we're saving. So, let's do `repo.tasks` is going to be equal to the task that gets passed in. Well, that'll just be our repo that we're going to use for right now. We're just going to save every task under its task ID back into our repo. Now that we have that, let's come down here into the `execute` and let's drop a push in here. So now, `repo.commands.push`, we're going to push the name of the command and whatever it is we passed in so that that way we're going to end up with a list of commands like `save` and `get` and all of those, and the object that was passed in. Now, for this one, we're just going to kind of keep it simple. Remember, `args` is going to pass us an array, and I just want the object that's passed in. So we're going to take the first one, `args` and drop it in there so that we have our object. Okay, now when I run this, I'm going to have a list of commands, and a list of objects. Let's build this out just a little bit more, and instead of just one, let's run four. So now I've got four executes creating four tasks. And just down at the bottom, we'll do a `console.log`, and we'll add in our `repo.tasks`. Now, let's run this again, and you'll see we have four tasks now in our database, our repository. Task one, two, three, and four, and the full task. Now, here's the beauty of the command pattern. I have in my commands object, my commands array, all of the things that have been done to this repo since it was created. And if something catastrophic were to happen, like I don't have a database connection, or an error happen, I actually have the ability to rerun those commands again in order to make this work. So let's play with that just a little bit so I can show you what that looks like. We're going to create a new method on our repo called `replay`. And that's going to be a function. And what `replay` is going to do is run over our commands, and re-execute them. Now, there's one issue with this in that I'm executing, and in my `execute` I'm logging it in my repo command, so let's actually create a new method called `repo.executeNoLog`, which is basically `repo`, just without this one `commands.push`. So we're executing it, but we're not logging it because we don't want to create an infinite loop here. All right, so now what's going to happen, recap real quick, we created a `replay` function that is basically just going to loop over the commands and rerun them. And we're going to call



executeNoLog in order to make this work. Now, let's come down to the bottom and let's try this out. So I'm going to log my tasks. Then I'm going to go `repo.tasks` equals empty. That's bad. I just deleted everything in my repository. And I'll prove that to you by logging this out again. Now, don't worry. I can do that because now I can do `repo.replay`. And all that's going to do is rerun all of the commands that were executed against my `repo`. And I'm going to run it one more time. I'll run my logging one more time, and you can see that this worked. So let's try this one more time. Let's run this, and what you're going to see is you're going to see the fullest of tasks, you'll see nothing, and then you'll see the replay of tasks. So look at this. Check this out. You've got saving four tasks. There's the four tasks, then you've got nothing. Then we ran our `repo.replay`, and look. We've got our saving four tasks again, and our four tasks are back in our repository. So what this command pattern allows me to do is log all of the commands that are executed against my object, and then rerun them as necessary. And actually, another thing we could do if we wanted to take the time, is I could do a `get` on my task. So if I save a task, I could do a `get` first just in my `save`, and store the old version in my commands. And then I could do a `rollback` that would then just save the old version. There's all kinds of things you can do with this command pattern when you start treating your commands as a full-blown object.

Summary

All right, we have made it. We have come a long way in our discussion on design patterns in JavaScript. This isn't all the patterns in the Gang of Four, but what I did is I picked out a bunch that are used in the JavaScript most often. There's probably some other ones that you might come across, but these are the majority of the JavaScript ones that are used. We started out by talking about objects and how objects work in JavaScript because they're a little bit different. They don't work exactly the same way as they do in everything else. So that was a good way for us to just level set on how objects worked. Then we talked about the three kinds of design patterns. Creational design patterns that were involved in creating new instances of an object, structural patterns, which deal with the makeup of objects and how objects look and interact with themselves, and then we ended with the talk of behavior patterns, and we looked at three patterns that help objects interact with each other, and how communications flow between objects. I hope you've enjoyed this course as much as I've enjoyed making it. This has been a fantastic course, I've had a lot of fun. Hopefully you've enjoyed it and you've gotten something out of it. I'd love to read your comments down in the discussion area. If you've got any, just drop a line down there and let me know if you've got any questions. Looking forward to the conversation.